

# Persistência com Spring Data

Utilização do framework Spring para persistência de dados em bases relacionais e NoSQL, com base em exemplos do PostgreSQL e do MongoDB dentro da arquitetura MVC.

Prof. Denis Cople



## 1. Itens iniciais

---

### Propósito

A criação de sistemas com persistência em bancos de dados PostgreSQL e MongoDB, por meio do Spring Data, e de APIs REST como meios para a manipulação e consulta aos dados.

### Preparação

Você precisa configurar o ambiente com a instalação do JDK e do Spring Tools Suite, além de fazer o download dos [projetos](#) desenvolvidos neste conteúdo.

### Objetivos

- Descrever o funcionamento do framework Spring e do Spring Data.
- Empregar o Spring na construção de serviços REST para bases relacionais.
- Empregar o Spring na construção de serviços REST para bases NoSQL.
- Aplicar o Spring Boot para a construção de aplicativo com acesso a serviços REST.

### Introdução

Neste conteúdo, vamos abordar os elementos de persistência do framework Spring com base em Spring Data e JPA, utilizando o Spring Tools Suite. Inicialmente, veremos as características básicas do Spring e aprenderemos a construir um sistema Web padrão, baseado em rotas e controladores, persistindo os dados em base H2.

Também demonstraremos a utilização do Postman para o teste de APIs REST e o uso de Retrofit para a comunicação com os serviços a partir do Java.

Teremos a definição de uma camada de integração, englobando clientes para todas as APIs desenvolvidas, e criaremos um aplicativo Web, demonstrando a importância dos elementos de interoperabilidade na definição de arquiteturas complexas que envolvem múltiplos aplicativos em ambientes heterogêneos.

1. Spring framework

# Características gerais

## Framework Spring e Spring Data

Assista ao vídeo e confira os principais pontos sobre este tópico.



### Conteúdo interativo

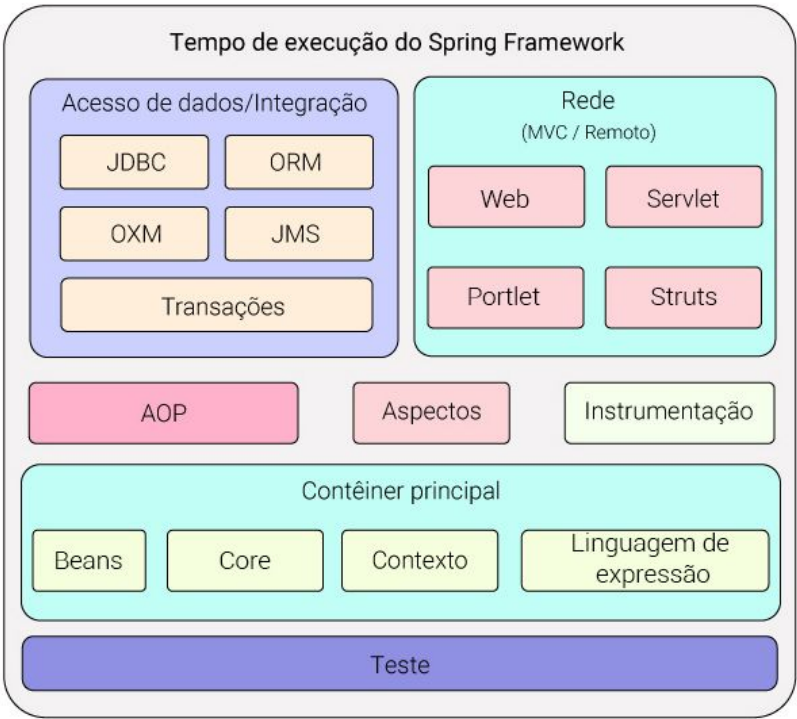
Acesse a versão digital para assistir ao vídeo.

## Modularização do framework

Baseado em **código aberto**, o framework **Spring** é uma ferramenta de grande utilização no mercado de desenvolvimento para **Web** com base na plataforma **Java**.

Além disso, implementa a **inversão de controle**, em que parte do fluxo de execução não é definido no código, mas efetuado a partir de um container no servidor. Ao utilizar essa abordagem, ocorre uma minimização do **acoplamento** no sistema e do esforço de programação necessário.

A imagem a seguir ilustra os componentes do framework **Spring**:



Módulos do framework Spring.

A base do núcleo é o **Core Container**, no qual temos as classes estruturais, suporte à injeção de dependências, acesso ao contexto de execução e comunicação com os componentes. Também é definida a sintaxe da Spring Expression Language (**SpEL**), utilizada na consulta e manipulação de componentes.

Para acesso a diferentes tipos de middleware, como bancos de dados, temos o **Data Access/Integration**, que também oferece suporte ao controle transacional e mapeamento objeto-relacional.

No módulo **Web**, temos componentes como Servlets e Portlets, para construir sistemas cliente-servidor baseados no protocolo HTTP, adotando sempre uma arquitetura MVC.

## Construção de componentes X configuração do contexto de execução

No Spring, temos uma divisão clara entre a construção de componentes e a configuração do contexto de execução. De forma geral, os **componentes** são definidos por meio de **classes anotadas**, enquanto as **configurações** necessárias são definidas por meio de arquivos no formato **XML**.

Podemos observar um arquivo de configuração utilizado pelo framework, no qual é preparado o **middleware** para a conexão com o **Derby Database**:

### Derby Database

Apache Derby é um sistema de gerenciamento de banco de dados relacional Java que pode ser embutido em programas Java e usado para processamento de transações on-line.

plain-text



Aqui, temos a configuração de um gestor de **persistência**. É necessário dar atenção ao uso de **namespaces**. Após incluir todos os namespaces necessários, há as configurações. A mais simples é o contexto, com a definição do pacote a partir do qual os **beans** (componentes) serão pesquisados. Adotamos o valor **exemplo01**.

A conexão com o banco de dados tem como base a classe **DriverManagerDataSource**. As propriedades necessárias são o driver do Derby, a URL de conexão, usuário e senha. A conexão é utilizada pela fábrica de gerenciadores, do tipo **LocalContainerEntityManagerFatoryBean**, fazendo a ponte com o driver JPA por meio da classe **EclipseLinkJpaVendorAdapter**. As **entidades** deverão ser criadas no pacote **exemplo01.model**, bem como os elementos que controlam a persistência.

Por trabalhar com o JPA, o controle transacional será baseado na classe **JpaTransactionManager**.

## Arquitetura de implementação

Observe que a camada **Model** será definida no pacote **exemplo01.model**, onde as entidades são criadas com o mapeamento **objeto-relacional** do **JPA**.

Na entidade **Produto**, a anotação **Entity** garante a persistência em uma tabela de mesmo nome, e **Id** é o campo **chave primária**. Veja:

java

```
@Entity
public class Produto {
    @Id
    public Integer codigo;
    public String nome;
    public Integer quantidade;

    public Produto() {}
    public Produto(Integer codigo, String nome, Integer quantidade){
        this.codigo = codigo;
        this.nome = nome;
        this.quantidade = quantidade;
    }
}
```

Implementaremos a persistência no padrão **DAO** (Data Access Object), segundo duas estratégias possíveis: a primeira seria uma **classe** definida como um **bean**, com a implementação dos métodos necessários, e a segunda utiliza uma interface **JpaRepository**, delegando para o Spring a criação desses métodos.

Tendo como base a primeira estratégia, os componentes de persistência são **classes anotadas**, que utilizam o middleware de forma **automática**, como na classe **ProdutoConcreteDAO**, criada no pacote **exemplo01.model**. Veja:

java

```
@Component
public class ProdutoConcreteDAO {
    @PersistenceContext
    private EntityManager em;

    public void save(Produto produto) {
        em.persist(produto);
    }
    public List findAll() {
        return em.createQuery("SELECT p FROM Produto p")
            .getResultList();
    }
}
```

Enquanto a anotação **Component** coloca a classe no contexto do Spring, ou seja, transforma-a em um **bean**, a anotação **PersistenceContext** obtém o gerenciador de entidades por meio da fábrica configurada anteriormente. O gerenciador será utilizado para consultar as entidades ou manipulá-las, como nos métodos **findAll** e **persist**.

Na segunda estratégia, definimos uma **interface** de persistência e deixamos o Spring cuidar do resto. A nova abordagem diminui o esforço de programação, já que as ações comuns são criadas automaticamente.

A interface **ProdutoDAO** demonstra a simplicidade do processo:

java

```
public interface ProdutoDAO
    extends JpaRepository {
}
```

Nossa interface descende de **JpaRepository**, tendo como parâmetros de tipo a classe **Produto**, que representa a entidade, e a classe **Integer**, para a chave primária, sendo criada no pacote **exemplo01.model**.

Agora, crie o controlador, utilizando o DAO e gerenciando as **transações**:

java

```
@Component
public class GerenciadorProduto {
    @Autowired
    private ProdutoDAO produtoDAO;

    @Transactional
    public void add(Produto produto){
        produtoDAO.persist(produto);
    }

    @Transactional(readOnly = true)
    public List findAll(){
        return produtoDAO.findAll();
    }
}
```

A classe **GerenciadorProduto** pertence à camada **Controller**, no pacote **exemplo01.controller**. Utilizando a anotação **Autowired**, configuramos automaticamente nosso DAO. **Transactional** define uma transação gerenciada pelo contexto, podendo ser configurada como somente leitura, quando não modificamos valores.

Agora, crie a **classe principal**, representada por:

java

```
public class ExemploSpring001 {
    public static void main(String[] args) {
        ClassPathXmlApplicationContext ctx =
            new ClassPathXmlApplicationContext("classpath:/spring.xml");
        GerenciadorProduto gerenciador =
            ctx.getBean(GerenciadorProduto.class);
        gerenciador.add(new Produto("A001","Caneta", 100));
        gerenciador.add(new Produto("A002","Lapis", 230));
        for(Produto p: gerenciador.findAll())
            System.out.println(p.getNome());
        ctx.close();
    }
}
```

O **contexto** do Spring é recuperado, a partir do arquivo de configuração, na variável **ctx**, o que nos permite obter os **beans**. Note que, ao instanciar **GerenciadorProduto**, teremos a inclusão automática de **ProdutoDAO**, garantindo a persistência.

# Spring Tools Suite

## Arquitetura MVC com Spring Framework

Assista ao vídeo e confira os principais pontos sobre este tópico.



### Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

## Instalação e criação do projeto Persistencia001

Podemos criar aplicativos Spring no **Spring Tools Suite**, com base no **Eclipse**, que será baixado a partir do endereço <https://spring.io/tools>, com a instalação via **jar** executável, efetuando uma simples extração de arquivos. Temos a inclusão das dependências no arquivo **pom.xml**.

Agora, crie o projeto na opção de menu **File (SpringToolSuite4.exe)**, seguida de **New** e **Spring Starter Project**, adotando o nome **Persistencia001**. Veja:

Service URL:

Name:

☒ Use default location

Location:

Type:  Packaging:

Java Version:  Language:

Group:

Artifact:

Version:

Description:

Package:

Criação de projeto Spring Boot.

Clicando no botão **Next**, selecione os pacotes **Spring Data JPA** e **H2 Database**, que serão adicionados ao arquivo de configuração **pom.xml**:

### H2 Database

Banco de dados Open Source, que funciona em memória, com um console acessível pelo browser dentro do contexto da aplicação.

Spring Boot Version:

Frequently Used:

☒ H2 Database ☒ Spring Data JPA ☐ Spring Web

☐ Thymeleaf

Available:

Selected:

☒ H2 Database

☒ Spring Data JPA

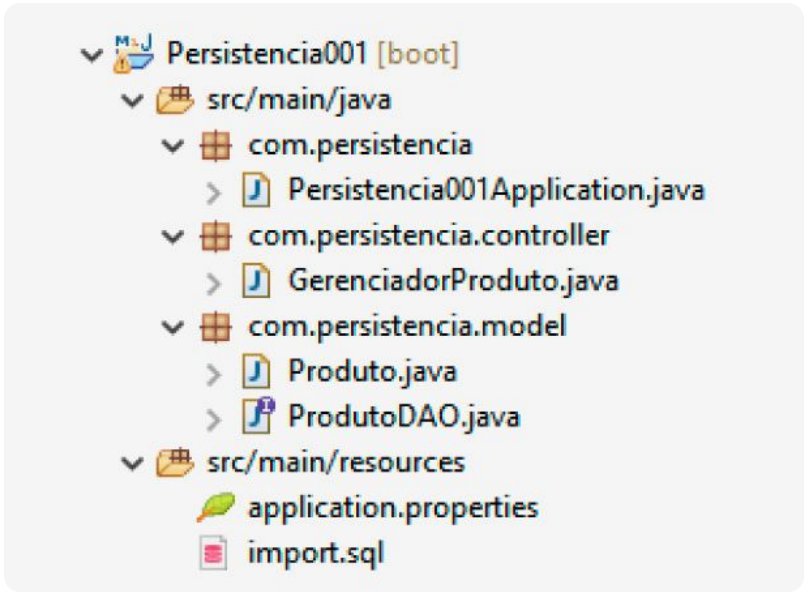
☒ H2 Database

Escolha dos pacotes utilizados no projeto.

Clique em **Finish**, e você terá um projeto **Spring Boot** bem estruturado, com inclusão do modelo objeto-relacional do JPA e banco de dados do tipo **H2**. Na execução padrão do Spring Boot, há um servidor na porta **8080**.

## Implementação das camadas Model e Controller

Precisamos cuidar das camadas **Model** e **Controller**, adicionando a classe **Produto** e a interface **ProdutoDAO**, descritas anteriormente, ao pacote **com.persistencia.model**, além de copiar a classe **GerenciadorProduto** para o pacote **com.persistencia.controller**. Vejamos:



Estrutura do projeto Persistencia001.

Observe que temos um banco em memória. Então precisamos alimentá-lo, criando o arquivo **import.sql**, em **src/main/resources**:

sql

```
INSERT INTO PRODUTO(CODIGO,NOME,QUANTIDADE) VALUES (1,'BANANA',1000);
INSERT INTO PRODUTO(CODIGO,NOME,QUANTIDADE) VALUES (2,'MORANGO',200);
INSERT INTO PRODUTO(CODIGO,NOME,QUANTIDADE) VALUES (3,'LARANJA',700);
```

Precisamos alterar a classe principal com o nome **Persistencia001Application**, em que é definido o aplicativo **Spring Boot**:

java

```
@SpringBootApplication
public class Persistencia001Application {

    @Autowired
    GerenciadorProduto controle;

    public static void main(String[] args) {
        SpringApplication.run(Persistencia001Application.class, args);
    }

    @Bean
    public CommandLineRunner commandLineRunner(ApplicationContext ctx){
        return args -> {
            System.out.println("<<< Verificação da base de dados >>>");
            controle.add(new Produto(101,"ABACATE",2000));
            for(Produto p: controle.findAll()) {
                System.out.println(p.nome);
            }
        };
    }
}
```

A classe **Persistencia001Application** já estava anotada como **SpringBootApplication** na versão inicial, caracterizando-a como um aplicativo Spring Boot, o qual é iniciado no método **main** com a chamada para **run**, da classe **SpringApplication**. Segundo o comportamento padrão, temos um servidor embarcado na porta **8080**.



## Comentário

Como queremos testar as camadas Model e Controller, precisamos do controlador no contexto do Spring, mediante a anotação **Autowired**, e de um direcionamento para a linha de comando. Para alterar a execução, precisamos do método **commandLineRunner**, anotado como **Bean**, assumindo um processamento sequencial.

Por meio da implementação **funcional**, o parâmetro **args** recebe uma sequência de comandos onde um produto é incluído, seguido da listagem dos nomes. O método **add** incluiu o produto, enquanto o método **findAll** retornou todos os produtos da base.

Teste o projeto: clique com o **botão direito** sobre ele no **Package Explorer**, com a divisão esquerda do ambiente, e escolha **Run As**, seguido da opção **Spring Boot App**:

```
Problems Javadoc Declaration Console X Progress
<terminated> Persistencia001 - Persistencia001Application [Spring Boot App] C:\Java\IDE\sts-4.14.0.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full\win32_x86_64_17.0.2\20220801-
2022-08-03 23:20:59.362 INFO 22360 --- [main] j.LocalContainerEntityManagerFactoryBean
2022-08-03 23:20:59.803 INFO 22360 --- [main] c.p.Persistencia001Application
<<< Verificação da base de dados >>>
BANANA
MORANGO
LARANJA
ABACATE
2022-08-03 23:21:00.037 INFO 22360 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean
```

Acompanhamento da execução no Console.

Temos uma camada **View** na forma de **linha de comando**. Redefiniremos a camada **View** no modelo **Web**, com base em páginas **HTML**.

# Rotas e controladores

Assista ao vídeo e confira os principais pontos sobre este tópico.

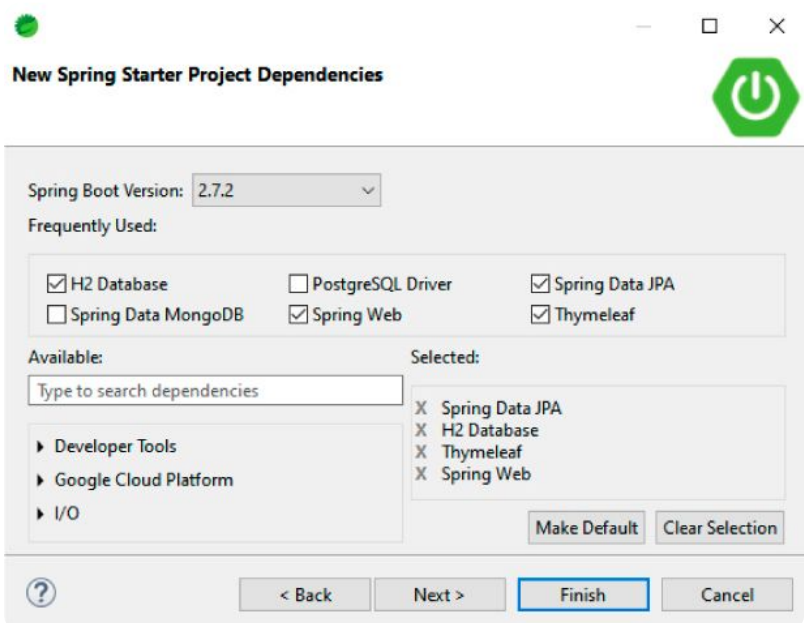
## Conteúdo interativo



Acesse a versão digital para assistir ao vídeo.

## Criação do projeto Persistencia002

Crie o projeto **Persistencia002**, com os pacotes **com.persistencia**, **H2 Database**, **Spring Data JPA**, **Spring Web** e **Thymeleaf**. Observe:



Bibliotecas utilizadas em Persistencia002.

A biblioteca **Spring Web** permitirá a criação de controladores HTTP, e as páginas para resposta serão definidas em modelos do **Thymeleaf**. Podemos efetuar um teste, executando o projeto como **Spring Boot Application**, e o servidor ficará ativo na porta 8080. Como não há **rotas** definidas, a chamada ao endereço de base (**localhost: 8080**) retornará um erro.

rotas

Endereços que iniciam ações nos controladores com subsequente redirecionamento para a página de resposta. Essa é uma estratégia adotada no Express, um servidor minimalista construído com NodeJS, sistemas Web com ASP.NET, e no Spring Boot, entre outros.

## Implementação dos controladores

Adicione o pacote **com.persistencia.model** com a classe **Produto** e a interface **ProdutoDAO**, do exemplo anterior, aproveitando também o arquivo **import.sql**, no diretório **resources**. Em seguida, crie **ProdutoWebController** no pacote **com.persistencia.controller**. Veja:

java

```
@Controller
@RequestMapping("/web/produtos")
@EnableTransactionManagement
public class ProdutoWebController {

    @Autowired
    ProdutoDAO dao;

    @RequestMapping(value="/dadosProduto", method=RequestMethod.GET)
    public ModelAndView dadosProduto() {
        return new ModelAndView("dadosProduto","produto", new Produto());
    }
    @RequestMapping(value = "/dadosProduto/{codigo}",
        method = RequestMethod.GET)
    public ModelAndView dadosProduto(
        @PathVariable(value="codigo") Integer codigo) {
        return new ModelAndView("dadosProduto","produto",
            dao.findById(codigo).get());
    }
    @RequestMapping(value="/dadosProduto",
        method=RequestMethod.POST)
    public String salvarProduto(
        @ModelAttribute Produto produto, ModelMap model) {
        dao.save(produto);
        model.addAttribute("produtos", dao.findAll());
        return "listaProdutos";
    }
    @RequestMapping(value="/removeProduto",
        method=RequestMethod.GET)
    public String delProduto(
        @RequestParam(name="codigo") Integer codigoProduto,
        ModelMap model) {
        dao.deleteById(codigoProduto);
        model.addAttribute("produtos", dao.findAll());
        return "listaProdutos";
    }
    @RequestMapping(value = "/listaProdutos",
        method = RequestMethod.GET)
    public String listaProdutos(ModelMap model) {
        model.addAttribute("produtos", dao.findAll());
        return "listaProdutos";
    }
}
```

A anotação **Controller** define um controlador Web capaz de responder às solicitações **GET** e **POST** que serão utilizadas. **EnableTransactionManagement** habilita transações via container do Spring, e **RequestMapping** define a rota inicial de acesso. Como atributo, há apenas uma instância de **ProdutoDAO**, colocada no contexto do Spring pela anotação **Autowired**.



## Atenção

Ao adotar EnableTransactionManagement, as transações serão gerenciadas pelo contexto do Spring de forma automática.

Cada método do controlador tem uma rota relativa para o acesso, bem como o método HTTP utilizado. Uma rota relativa é complementada pela inicial, como no método **listaProdutos**, com a rota de mesmo nome, resultando em **http://localhost:8080/web/produtos/listaProdutos**.



## Comentário

Há dois métodos sobrecarregados com o nome dadosProduto, ambos com rotas de mesmo nome, em modo GET, mas, no segundo, a rota é complementada com o código do produto. Para ambos, teremos a apresentação de um formulário HTML baseado em um template que também recebe o nome dadosProduto e que é preenchido com uma entidade vazia, no caso da inserção, ou com os dados da entidade atual, no caso da alteração.

Observe o retorno utilizado, do tipo **ModelAndView**, em que temos o nome do **template**, **apelido** da entidade que será recuperada, e a **instância** da entidade. No caso da inserção, há apenas uma instância vazia, enquanto na alteração ocorre a consulta via **findById**, com o código da rota, recuperado via **PathVariable**.

No método **salvarProduto**, que também responde na rota **dadosProduto**, mas, agora, via **POST**, fornecemos a resposta para o formulário anterior. Os dados enviados são recuperados no objeto produto devido ao uso de **ModelAttribute**, permitindo persistir a entidade recebida por meio do método **save**.



## Comentário

A lista de produtos, obtida via `findAll`, é fornecida com o apelido `produtos` para o template por meio de um `ModelMap`. Para esse método, o retorno é o nome do template, que se chamará `listaProdutos`.

O método para exclusão, que recebeu o nome **`delProduto`**, responderá na rota **`removeProduto`**, utilizando o modo **`GET`**, e terá como parâmetro da requisição o código da entidade. O parâmetro da requisição é associado a **`codigoProduto`** através de **`RequestParam`**, causando a exclusão do registro na chamada para **`deleteById`**. O restante do método preenche a lista de entidades e direciona para o template **`listaProdutos`**.

Nosso último método tem o nome **`listaProdutos`**, respondendo na rota de mesmo nome, via modo **`GET`**, com a recuperação dos produtos e direcionamento para o template. Note que ele corresponde ao final do fluxo de execução dos métodos `salvarProduto` e `delProduto`.

# Interface com Thymeleaf

## Sintaxe básica

Em nosso sistema, como estamos adotando **`Thymeleaf`**, serão utilizados templates HTML para a construção da interface. Precisamos entender os **`símbolos`** usados no Thymeleaf, segundo o padrão da **`Standard Expression Syntax`**. Vejamos a seguir os símbolos utilizados na sintaxe do Thymeleaf:

**`${...}`**

Acessa o valor de uma variável, que pode ser local ou fornecida no Model.

**`*{...}`**

Seleciona o campo de um objeto.

**`#{...}`**

Recupera mensagens configuradas em um arquivo com extensão `properties`.

**`@{...}`**

Transforma o endereço da rota em uma URL válida.

**`~{...}`**

Insere o conteúdo de um fragmento HTML, viabilizando reuso no template.

Uma característica positiva do Thymeleaf é a possibilidade de criar **`templates`** de fácil integração com Spring na sintaxe **`HTML`**.

## Página inicial

Para resolver o problema na chamada para a **`raiz`**, vamos adicionar a classe **`HomeController`** ao pacote **`com.persistencia.controller`**:

java

```
@Controller
public class HomeController {

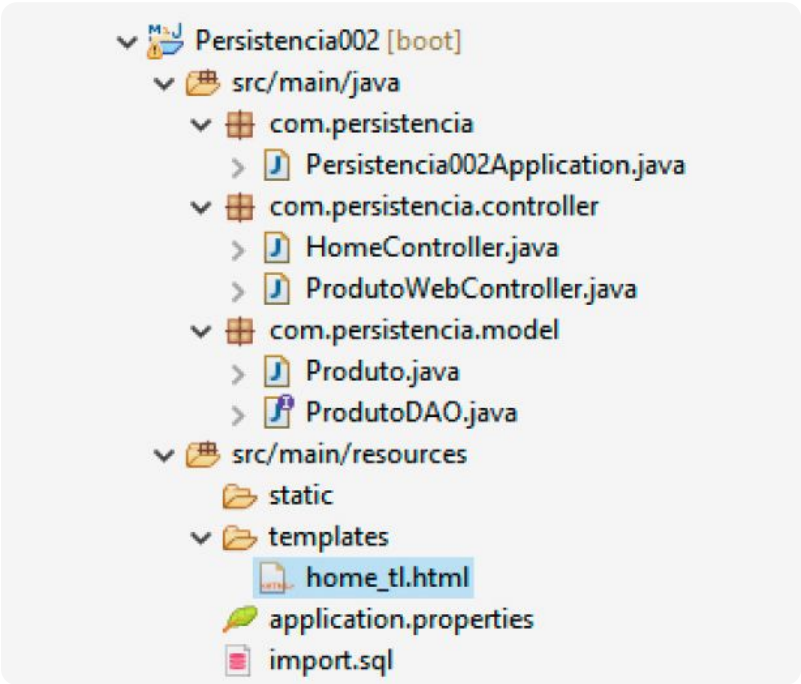
    @GetMapping("/")
    public String showHome() {
        return "home_tl";
    }
}
```

Temos o direcionamento para o template **home\_tl** quando a **raiz** do site é requisitada no modo **GET**. Para criar o template, utilizamos o botão direito sobre o diretório **templates**, dentro de **resources**, seguido de **New** e **File**. Nesse caso, adotamos o nome **home\_tl.html**. A extensão deve ser digitada, e o conteúdo do arquivo é apresentado a seguir:

plain-text

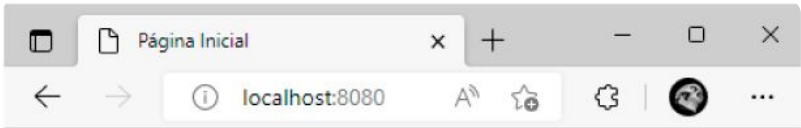
# Produtos

No início do template, utilizamos o namespace **th** para a inclusão da sintaxe empregada pelo **Thymeleaf**. A página conta apenas com um link para a listagem de produtos, no qual a rota é fornecida via **th:href** e corrigida pelo uso de **arroba**. Veja:



Estrutura parcial do projeto Persistencia002.

Teste a exibição da página inicial pela opção **relaunch**, na barra de ferramentas da IDE, com um ícone mesclando os símbolos **stop** e **run**. Acesse o endereço **http://localhost:8080**:



## Produtos

Página inicial do sistema Persistencia002.

## Implementação da interface Lista Produtos

Os controladores sempre fazem a associação com um modelo (**template**) para a construção da resposta no **retorno** do método, ou como primeiro parâmetro do construtor de um **ModelAndView**.

Inicie a implementação dos modelos, criando os arquivos no diretório **templates**, a começar por **listaProdutos.html**. Observe:

plain-text

# Produtos

Novo Produto

valor

Código: valor

Quantidade: valor

O primeiro link aponta para a rota `/web/produtos/dadosProduto`, sem a complementação com o código, para iniciar o formulário em modo de inclusão. Em seguida, temos uma divisão da classe `container`, responsável por organizar os painéis que exibirão os dados.

Para criar os painéis de dados, usamos `th:each`, com a definição da variável `produto`, que recebe cada uma das entidades do conjunto `produtos`, fornecido pelo controlador. Para cada ocorrência, temos o valor dos campos em elementos do tipo `span`.

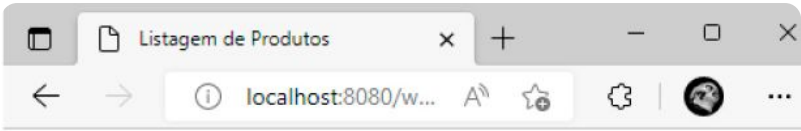


## Comentário

O texto é substituído pelo valor do campo, com a utilização de `th:text`, tendo ainda os links para a exclusão, com o código fornecido pelo parâmetro na requisição, e para a inclusão, em que acrescentamos o código do produto ao final da rota, por meio da variável de associação com o nome `cod`. Note que o uso de arroba é necessário para a conversão de todos os endereços.

Com a utilização de uma folha de estilos, o visual da página ficará muito fluido, baseado em um container `flex`, que organizará as divisões de dados segundo um fluxo horizontal, com a quebra ocorrendo de acordo com as dimensões da janela.

Para as divisões de dados, foi definida a classe `pnlProduto`. Teste a listagem antes, reinicie o servidor e clique sobre o link da página inicial. Veja:



## Produtos

Novo Produto

|                                                            |                                                            |
|------------------------------------------------------------|------------------------------------------------------------|
| BANANA<br>Código: 1<br>Quantidade: 1000<br>Excluir Alterar | MORANGO<br>Código: 2<br>Quantidade: 200<br>Excluir Alterar |
| LARANJA<br>Código: 3<br>Quantidade: 700<br>Excluir Alterar |                                                            |

Tela de listagem de produtos.

## Implementação da interface Cadastro de Produto

Agora, vamos definir a tela de cadastramento, mas, antes, precisamos alterar a classe **Produto**, encapsulando seus atributos por meio de **getters** e **setters** – uma exigência para os **formulários**. Vamos lá:

```
java
@Entity
public class Produto {
    @Id
    private Integer codigo;
    private String nome;
    private Integer quantidade;

    public Produto(){}
    public Produto(Integer codigo, String nome, Integer quantidade){
        this.codigo = codigo;
        this.nome = nome;
        this.quantidade = quantidade;
    }

    public Integer getCodigo() { return codigo; }
    public void setCodigo(Integer codigo) { this.codigo = codigo; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public Integer getQuantidade() { return quantidade; }
    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
    }
}
```

Agora, vamos definir o modelo para cadastro, criando o arquivo **dadosProduto.html** no diretório **templates**:

plain-text

Código

codigo

Nome

Quantidade

Salvar

Aqui, temos um formulário integrado ao Spring, com base no objeto **produto**, que foi enviado a partir do controlador e recuperado através de **th:object**. A rota de destino para os dados será **/web/produtos/dadosProduto**, no modo **POST**, associada via **th:action** e corrigida para o endereço certo com a utilização de **arroba**.

Comentário

Definido o objeto do formulário, os campos de entrada são associados a seus atributos por meio da notação de asterisco, com a especificação do atributo entre chaves, juntamente com o uso de th:field. A configuração utilizada aqui efetuará o preenchimento automático dos dados, bem como associará corretamente os nomes dos campos para envio ao servidor.

A chave primária não pode ser alterada. Como não há autoincremento, ela será digitada apenas na inserção, levando a uma exibição condicional. Por meio do uso de **th:if**, determinamos que um **input** do tipo **number** será exibido se o código do produto for nulo (**inserção**), enquanto para a **alteração** teremos o **span** contendo um elemento **hidden** e o código obtido via **th:text**.

Aplicando o CSS, temos o formulário a seguir:

Dados do Produto

Código

4

Nome

ABACAXI

Quantidade

750

Salvar

Dados do Produto

Código

2

Nome

MORANGO

Quantidade

200

Salvar

Tela de cadastro nos modos de inserção e alteração.

# Verificando o aprendizado

Questão 1

A arquitetura MVC é considerada um padrão de grande relevância no desenvolvimento de aplicativos cadastrais e é organizada de forma estrutural no framework Spring. Segundo o padrão arquitetural, qual camada deveria conter os componentes do tipo DAO?

A

Controller

B

Abstraction

C

Presentation

D

View

E

Model



A alternativa E está correta.

Como o DAO concentra as operações sobre o banco de dados, a camada Model, relacionada à persistência, seria a correta. A camada Controller concentra os objetos de negócio, e a camada View agrupa os elementos da interface de usuário. As camadas Presentation e Abstraction fazem parte de outro padrão arquitetural, denominado PAC (Presentation, Abstraction e Controller).

Questão 2

O framework Spring permite a inclusão de funcionalidades nas classes de forma simples, com base em anotações, as quais são reconhecidas pelo ferramental, gerando o código necessário para obter o funcionamento desejado. Qual anotação coloca o componente DAO no contexto de execução do Spring?

A

Controller

B

Autowired

C

Component

D

RequestMapping

E

Transactional



A alternativa B está correta.

Quando utilizamos Component para anotar uma classe, temos um elemento que pode se relacionar com o contêiner facilmente, por meio da anotação Autowired, que coloca um componente qualquer, como o DAO, no contexto de execução do Spring. Controller transforma a classe em um controlador, com as rotas para seus métodos mapeadas através de RequestMapping. Para habilitar o gerenciamento de transações, nosso controlador deve ser anotado com EnableTransactionManagement.



## 2. Criação de um servidor REST para manipulação de dados



# Banco de dados PostgreSQL

## Modelo relacional

Na prática, um banco de dados relacional é um **repositório**. Utilizando a SQL, vamos criar a tabela **produto**:

```
sql

CREATE TABLE PRODUTO(
  CODIGO INTEGER NOT NULL PRIMARY KEY,
  NOME VARCHAR(20),
  QUANTIDADE INTEGER);
```

Após a definição da estrutura de armazenamento, podemos utilizar os comandos para a **manipulação** de **registros**, como na **inclusão** do produto, efetuada por meio do **INSERT**. Veja:

```
sql

INSERT INTO PRODUTO (CODIGO, NOME, QUANTIDADE)
VALUES (2, "Morango", 200);
```

No ambiente de programação orientado a objetos, é necessário efetuar o mapeamento **objeto-relacional**, em que os conjuntos de registros são expressos como coleções de objetos – técnica que já utilizamos ao trabalhar com **Spring Data** e **JPA**.

## Instalação do PostgreSQL

## Instalação do PostgreSQL e PGAdmin

Assista ao vídeo e confira os principais pontos sobre este tópico.





### Conteúdo interativo

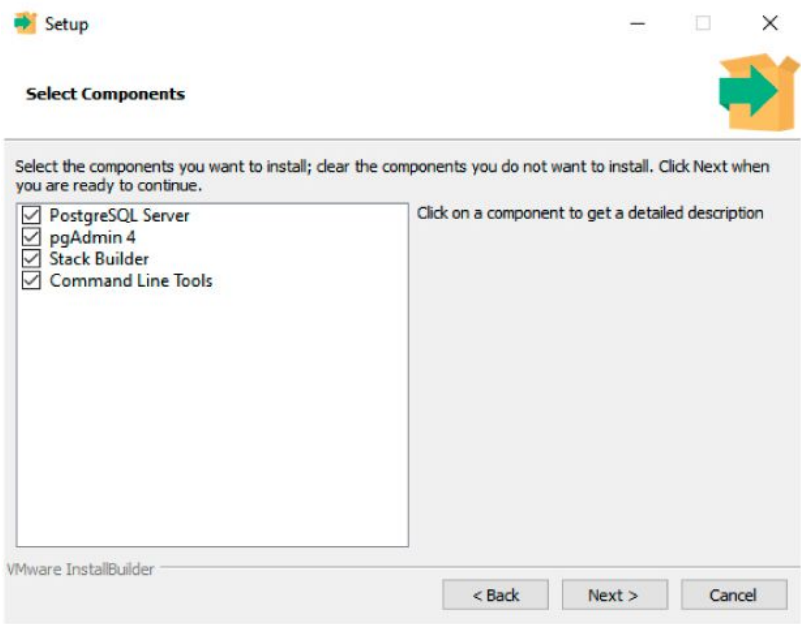
Acesse a versão digital para assistir ao vídeo.



Por que o PostgreSQL? Temos um servidor de **código aberto** com ampla utilização no mercado, que apresenta **opções avançadas**, como gatilhos e procedimentos armazenados.

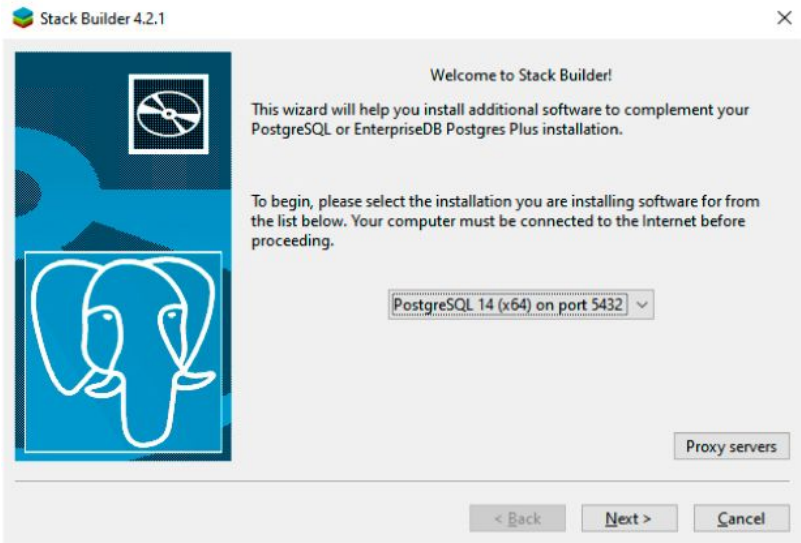
Acesse o endereço <https://www.postgresql.org/>, selecione um link para download de acordo com o sistema operacional, além da versão – aqui, utilizamos a 14.4.

Durante a instalação, deve ser definida a senha do usuário **postgres** – sugerimos **admin123** – bem como a acentuação padrão, que será modificada para "**Portuguese, Brazil**". Observe:



Tela do instalador gráfico para o PostgreSQL.

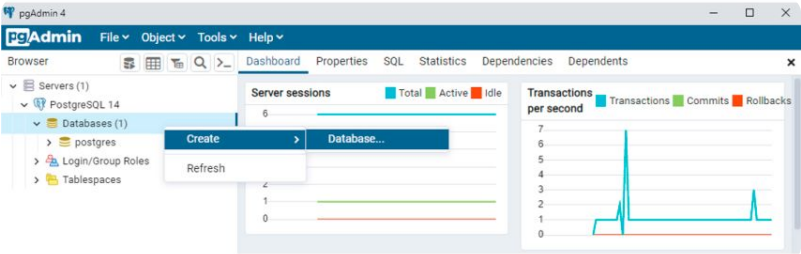
No **Stack Builder**, selecione o servidor que foi instalado, clique em **Next** e selecione a opção **pgJDBC**, na divisão **Database Drivers**, para fornecer suporte ao Java. Veja:



Tela do instalador de complementos do PostgreSQL.

## Ferramenta PGAdmin

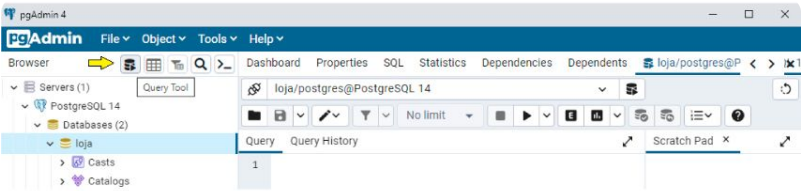
A ferramenta administrativa do PostgreSQL é o **pgAdmin**. Para criar o banco, clique com o botão direito na divisão **Databases**, escolha a opção **Create**, seguida de **Database**, e forneça o nome do banco – que, no caso, será **loja**:



Criação do banco de dados no pgAdmin.

Utilize o usuário **postgres**. Com o banco **loja** selecionado na árvore de navegação (divisão **Databases**), escolha **Query Tool**, digitando, em seguida, o comando para criação de **PRODUTO**, do tópico anterior, e executando com **F5**.

Podemos acionar o **Query Tool** a partir do menu **Tools**, utilizando o clique do botão direito sobre **loja** ou clicando no **ícone** da barra de ferramentas:



Indicação do ícone de abertura do Query Tool.

Depois, utilize o comando **INSERT** para alimentar a tabela com alguns dados ou o conteúdo do arquivo **import.sql**, sempre com o uso de **F5** para execução. Por fim, utilize o comando **SELECT** para verificar a inclusão dos dados.

Agora, crie a tabela **MOVIMENTO** para registrar as movimentações dos produtos:

```
sql

CREATE TABLE MOVIMENTO (
  ID INTEGER NOT NULL PRIMARY KEY,
  CODIGO INTEGER NOT NULL REFERENCES PRODUTO(CODIGO),
  QUANTIDADE INTEGER,
  TIPO CHAR(1),
  DIA DATE);
```

O campo Tipo pode ser **E** (Entrada) ou **S** (Saída).

# Tecnologia REST

## Criação do projeto Persistencia003

Vamos criar outro projeto, do tipo **Spring Starter Project**, adotando o nome **Persistencia003** e pacote **com.persistencia** – mesmo padrão utilizado nos exemplos anteriores. Para o novo projeto, selecionaremos as bibliotecas **Spring Data JPA**, **Spring Web** e **PostgreSQL Driver**.

Crie o pacote **com.persistencia.model** no novo projeto, e copie para ele as classes **Produto** e **ProdutoDAO** do projeto **Persistencia002**. Posteriormente, nossa classe **Produto** sofrerá uma modificação, devido ao relacionamento com a nova classe, com o nome **Movimento**, que deverá ser criada no pacote **com.persistencia.model**. Observe:

```
java

@Entity
@JsonIgnoreProperties({"hibernateLazyInitializer","handler",
                    "produto"})
public class Movimento {
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE,
                    generator = "idGen")
    @TableGenerator(name = "idGen", table = "TAB_IDS")
    private Integer id;

    @ManyToOne
    @JoinColumn(name = "codigo")
    private Produto produto;

    private Integer quantidade;
    private Character tipo;
    private Date dia;

    // Getters e Setters
    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public Produto getProduto() { return produto; }
    public void setProduto(Produto produto) {
        this.produto = produto;
    }
    public Integer getQuantidade() { return quantidade; }
    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
    }
    public Character getTipo() { return tipo; }
    public void setTipo(Character tipo) { this.tipo = tipo; }
    public Date getDia() { return dia; }
    public void setDia(Date dia) { this.dia = dia; }
}
```

A nova entidade utiliza uma chave **autoincrementada**, no campo inteiro **id**, adotando estratégia baseada em tabela. O gerador, de nome **idGen**, define a tabela **TAB\_IDS** como repositório de identificadores, por meio da anotação **TableGenerator**, ocorrendo a associação com o campo via **GeneratedValue**. Essa abordagem independe do tipo de banco de dados, já que não utiliza recursos de incremento ao nível da DDL ou sequências.

Temos o **relacionamento** com a tabela **PRODUTO** por meio de um campo do tipo da entidade, com o nome **produto**. A anotação **ManyToOne** define a relação vista pelo lado do movimento, com a escolha do campo que determina o valor via **JoinColumn**.

Há ainda uma anotação **JsonIgnoreProperties** para evitar a recursividade na relação bilateral, na qual instruímos o Hibernate (driver utilizado pelo JPA) a ignorar o campo **produto** nas consultas com retorno no formato **JSON** – algo que será necessário para implementar as respostas via **REST**.

Altere a classe **Produto** de forma a incluir o relacionamento, acrescentando o campo **movimentos**:

```
java

@Entity
@JsonIgnoreProperties({"hibernateLazyInitializer","handler",
    "movimentos"})
public class Produto {
    @Id
    private Integer codigo;
    private String nome;
    private Integer quantidade;

    // Inclusão do relacionamento - Inicio
    @OneToMany(fetch = FetchType.LAZY, mappedBy = "produto")
    private List movimentos;
    public List getMovimentos(){ return movimentos; }
    // Inclusão do relacionamento - Final

    public Produto(){ }
    public Produto(Integer codigo, String nome, Integer quantidade){
        this.codigo = codigo;
        this.nome = nome;
        this.quantidade = quantidade;
    }

    // Getters e Setters
    public Integer getCodigo() { return codigo; }
    public void setCodigo(Integer codigo) { this.codigo = codigo; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public Integer getQuantidade() { return quantidade; }
    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
    }
}
```

A relação exige uma anotação **OneToMany** sobre uma coleção de entidades do tipo **Movimento**, referenciada no campo **movimentos** e mapeada para o campo **produto** dos dependentes. Com a configuração **LAZY**, os valores são carregados apenas quando solicitados, e o encapsulamento em um **getter** garantirá que teremos somente leitura para a coleção. Novamente, utilizamos **JsonIgnoreProperties**, agora apontando para a coleção do relacionamento.

Precisamos de um DAO para Movimento, que será criado em **com.persistencia.model**, com o nome **MovimentoDAO**, descendendo de **JpaRepository**:

```
java

public interface MovimentoDAO
    extends JpaRepository{
}
```

Você precisa alterar a conexão padrão com o banco de dados em **application.properties**, no diretório **resources**:

```
java

spring.jpa.database=POSTGRESQL
spring.sql.init.platform=postgres
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:postgresql://localhost:5432/loja
spring.datasource.username=postgres
spring.datasource.password=admin123
```

Atenção para sua password do postgreSQL.

Além das duas opções iniciais, que definem o servidor como **PostgreSQL**, temos a exibição dos comandos SQL no console do servidor (**show-sql**), estratégia para criação de tabelas (**ddl-auto**) escolhida como **update**, url de conexão apontando para **loja**, no servidor local, nome do usuário e senha.



## Comentário

Algo que merece cuidado é a estratégia de criação, na qual o uso de create apaga e cria as tabelas a partir dos campos das entidades JPA, update cria as estruturas inexistentes, e none apenas utiliza as tabelas do banco. Podemos executar o servidor, que não terá páginas de acesso ainda, mas permitirá verificar se alguma configuração relacionada ao banco ou às entidades JPA está incorreta, já que o console apresentaria os erros.

Como as tabelas **PRODUTO** e **MOVIMENTO** já existem no banco de dados, apenas a tabela de identificadores, com o nome **TAB\_IDS**, será gerada na execução, podendo ser verificada com um comando **SELECT** sobre a nova tabela no **pgAdmin**.

## Fundamentos do REST

### Fundamentos do REST e a ferramenta Postman

Assista ao vídeo e confira os principais pontos sobre este tópico.

## Conteúdo interativo



Acesse a versão digital para assistir ao vídeo.

Ao contrário de um aplicativo Web, um serviço Web ou **Web Service** visa apenas o trânsito de dados entre cliente e servidor, sem a construção de uma interface de usuário.

Os serviços do tipo **SOAP** (Simple Object Acces Protocol) se baseiam no protocolo de mesmo nome, com uso do formato **XML**.

Já no tipo **RESTful**, os métodos do HTTP são utilizados para representar as operações efetuadas, segundo a arquitetura **REST** (Representational State Transfer), em que as entidades devem ser transmitidas no formato **JSON**.

Para o REST, **GET** é utilizado para efetuar **consultas**, podendo ocorrer o retorno de uma entidade ou uma coleção.



## Exemplo

O endereço `http://localhost:8080/produto` retornaria todos os produtos da base, enquanto `http://localhost:8080/produto/101` forneceria apenas o produto cujo código tem valor 101, devido ao acréscimo da chave no final do endereço.

Na exclusão, é utilizado o método **DELETE**, com o identificador anexado ao final do endereço; ou seja, a chamada **`http://localhost:8080/produto/101`**, adotando o modo **DELETE**, removeria o produto com código **101** da tabela. A captura do segmento final da rota, com o valor da chave, segue o padrão adotado anteriormente, com a utilização de **PathVariable**.



## Atenção

Tanto a inclusão quanto a alteração precisam de uma entidade fornecida no formato JSON, no corpo da chamada, sendo recuperada no controlador através de um parâmetro anotado como `RequestBody`.

Em algumas implementações do REST, ambas as ações utilizam o endereço de base do serviço, ou seja, **`http://localhost:8080/produto`**. O método **POST** é utilizado para a inclusão de uma entidade, e **PUT**, para efetuar uma alteração. Mas outras incluem o identificador, no segmento final da chamada, para a alteração, da mesma forma que ocorre para a remoção, trazendo uma **semântica** mais adequada.

A seguir observe as rotas de um serviço no modelo REST:

|                                                                                                   |
|---------------------------------------------------------------------------------------------------|
| /produto                                                                                          |
|                                                                                                   |
|                                                                                                   |
| Modo: GET<br>Objetivo: Consulta<br>Exemplo de chamada: <code>http://localhost:8080/produto</code> |

/produto/{codigo}

---

Modo: GET  
Objetivo: Consulta  
Exemplo de chamada: http://localhost:8080/produto/101

/produto

---

Modo: POST  
Objetivo: Inclusão  
Exemplo de chamada: http://localhost:8080/produto

/produto/{codigo}

---

Modo: PUT  
Objetivo: Alteração  
Exemplo de chamada: http://localhost:8080/ produto/101

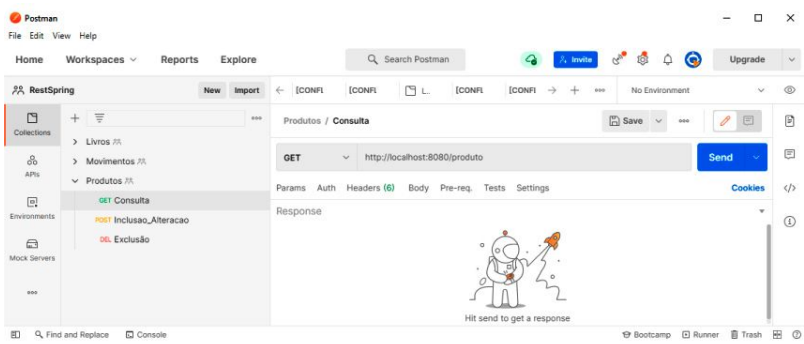
/produto/{codigo}

---

Modo: DELETE  
Objetivo: Exclusão  
Exemplo de chamada: http://localhost:8080/ produto/101

## Ferramenta para Web Services RESTful – Postman

Um aplicativo interessante para Web Services RESTful é o **Postman**, que permite o uso de **todos** os métodos do HTTP. Ele deverá ser baixado no endereço **<https://www.postman.com/downloads>**.



Teste de serviços por meio do aplicativo Postman.

Ao trabalhar com Postman, o primeiro passo é a criação de uma área de trabalho (**Workspace**), agrupando coleções de requisições. Após abrir o Workspace, devem ser definidas as **coleções**, adicionando as **requisições** relacionadas a cada uma delas.

Ao nível de uma chamada específica, podemos efetuar as configurações necessárias, a começar pelo **método** HTTP utilizado, e clicar no botão **Send** para executar a requisição.



## Comentário

O conceito de API ou Application Programming Interface pode ser descrito como um conjunto de funcionalidades oferecidas pelo sistema para programadores que desejam utilizá-las, o que garante o acesso controlado a determinadas funcionalidades.

Quando definimos um **Web Service**, é comum termos um grande sistema por trás da interface de chamadas disponibilizada, permitindo o uso das informações e serviços sem a exposição dos métodos internos. Se os Web Services são do tipo **RESTful**, estamos construindo uma **API REST**.

Você precisa compreender a sintaxe JSON! Por exemplo, os produtos da base de dados poderiam ser expressos no seguinte documento **JSON**, no qual a sintaxe nos fornece uma interpretação natural da estrutura:

```
python
```

```
{
  "produtos": [
    {"codigo": 1, "nome": "Lápis", "quantidade": 500},
    {"codigo": 2, "nome": "Caneta", "quantidade": 350},
    {"codigo": 3, "nome": "Borracha", "quantidade": 400}
  ],
  "total": 3
}
```

# API REST com o banco de dados PostgreSQL

## API REST para o PostgreSQL

Assista ao vídeo e confira os principais pontos sobre este tópico.

# Conteúdo interativo



Acesse a versão digital para assistir ao vídeo.

## Definição da camada Controller

Agora, vamos à camada **Controller**! Inicie pela criação de **ProdutoService**, no pacote **com.persistencia.controller**:

java

```
@RestController
@EnableTransactionManagement
public class ProdutoService {
    @Autowired
    ProdutoDAO dao;

    @GetMapping("/produto")
    public List obterTodos(){
        return dao.findAll();
    }
    @GetMapping("/produto/{codigo}")
    public Produto obterProduto(
        @PathVariable(value="codigo") Integer codigo){
        return dao.findById(codigo).get();
    }
    @PostMapping("/produto")
    public void incluir(@RequestBody Produto produto){
        dao.save(produto);
    }
    @PutMapping("/produto/{codigo}")
    public void alterar(
        @PathVariable(value="codigo") Integer codigo,
        @RequestBody Produto produto){
        if( (dao.findById(codigo).get()!=null) &&
            (produto.getCodigo()==codigo) )
            dao.save(produto);
    }
    @DeleteMapping("/produto/{codigo}")
    public void remover(
        @PathVariable(value="codigo") Integer codigo){
        dao.deleteById(codigo);
    }
}
```

Ao utilizar **RestController**, definimos a classe como um Web Service **RESTful**, permitindo mapear rotas para seus métodos. O retorno desses métodos é transformado automaticamente para **JSON**.

A transformação de JSON para Java, e vice-versa, atua sobre propriedades e atributos **públicos**. Nesse caso, é aconselhável o uso de **getters** e **setters**, de acordo com as boas práticas de programação, mas o uso de atributos simples não impediria a conversão necessária.



## Atenção

Para transformar atributos em propriedades, podemos clicar com o botão direito sobre o código, no editor, e escolher Source, seguido de Generate Getters and Setters.

Como os métodos **obterTodos** e **obterProduto** são de consulta, foram configurados para o modo **GET**. O primeiro retorna o conjunto completo de produtos, por meio de uma chamada para **findAll**, enquanto o segundo retorna apenas um produto.

A rota de **obterProduto** inclui o código no segmento final do endereço, com a captura do valor através da anotação **PathVariable**, permitindo a utilização na chamada para **findById**.

O método **remover** utiliza a mesma rota de **obterProduto**, mas, agora, no modo **DELETE** do protocolo HTTP. Da mesma forma que para o método anterior, o código do produto é capturado e utilizado na remoção da base por meio da chamada para **deleteById**.



## Comentário

Como o JpaRepository permite tratar a alteração e a inclusão usando o mesmo comando – que, no caso, é o save –, os métodos alterar e incluir capturarão a entidade fornecida no corpo da requisição, por meio da anotação RequestBody, e invocarão o comando citado.

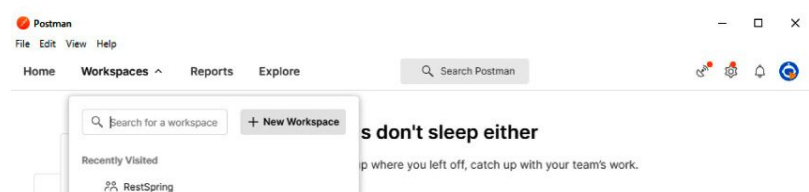
Mas enquanto a inclusão ocorre na rota de base, com uso do modo **POST**, na alteração temos o código do produto no segmento final do endereço, no modo **PUT**. O código fornecido servirá apenas para validar a entidade na alteração, antes de persistir, evitando que ocorra uma inclusão indevida.

Os métodos para manipulação de dados são do tipo **void**, ou seja, não retornam valores para o cliente, mas a execução bem-sucedida retorna uma resposta de código **200** ou **HTTP OK**.

Quanto aos métodos de consulta, no modo **GET** a resposta inclui os valores de retorno no corpo, adotando o formato **JSON**.

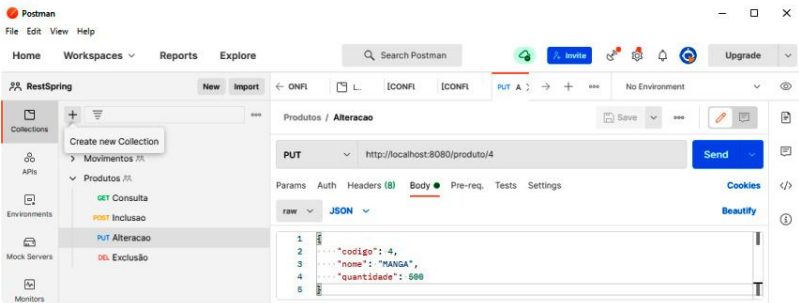
## Teste do serviço utilizando Postman Coleção de requisições Produtos

Crie no Postman o grupo de trabalho **RestSpring**, em **Workspaces**, no qual utilizamos a opção **New Workspace**:



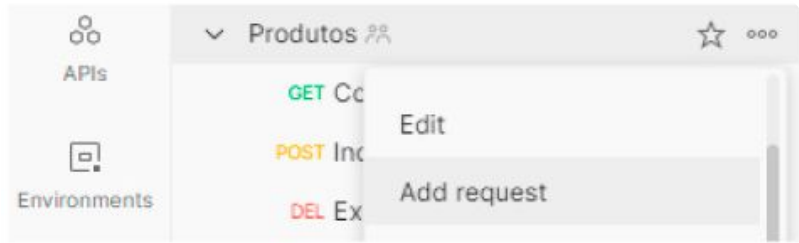
Gerência de Workspaces no Postman.

Após, ative-o e crie uma coleção de requisições com o nome **Produtos**, clicando no sinal de **adição**:



Gerência de coleções de requisições no Postman.

A criação da requisição é efetuada com a escolha da coleção **Produtos**. Clique no botão para opções adicionais, com o símbolo de **reticências**, seguido da opção **Add request**:



Adição de requisição na coleção.

Para as chamadas de consulta e exclusão, toda informação necessária estará no endereço de acesso, enquanto inserção e alteração exigirão o fornecimento dos dados, no formato **JSON**, através do corpo da requisição.

Como dados de teste, consideraremos o fragmento de código a seguir, que representa um **produto**, segundo a sintaxe **JSON**. Criaremos uma requisição com o nome **Inclusão**, apontando para o endereço **http://localhost:8080/produto**, no modo **POST**, e digitaremos os dados de nossa entidade na divisão **Body**, escolhendo o formato **raw** e tipo **JSON**. Veja:

```
plain-text

{
  "codigo": 4,
  "nome": "MANGA",
  "quantidade": 999
}
```

Após iniciar a execução de nosso servidor (no Spring Tool – Persistencia003), basta efetuar a chamada da nova requisição, com o clique no botão **Send**. Teremos como retorno apenas uma resposta do tipo **Http 200 Ok** e um corpo vazio. É possível verificar a inclusão no PostgreSQL por meio do **pgAdmin**.

Vamos criar a requisição com o nome **Consulta**, baseada no endereço **http://localhost:8080/produto**, modo **GET**, sem dados no corpo. Clicando no botão **Send**, teremos o retorno de todos os produtos, no formato **JSON**. Para obter um produto específico, basta adicionar o código ao endereço.

Com relação à alteração, podemos criar uma requisição com o nome **Alteração**, no modo **PUT**, apontando para **http://localhost:8080/produto/4**, com a mesma configuração de **Inclusão** na divisão **Body**. Altere algum dos valores do código JSON original, clique no botão **Send**, e faça uma nova consulta para verificar como a alteração foi efetivada no banco de dados.

Finalmente, a exclusão pode ser testada a partir de uma requisição com o nome **Exclusão**, que apontará para **http://localhost:8080/produto/4**, no modo **DELETE**, causando a remoção do produto de teste adicionado anteriormente.



## Atenção

Salve cada requisição, clicando no botão Save.

## Coleção de requisições Movimentos

Agora, vamos tratar da movimentação de produtos. Para isso, crie a classe de dados **MovimentoVO** para sinalizar o sucesso ou a falha de cada operação de movimentação, no pacote **com.persistencia.controller**:

```
java

public class MovimentoVO {
    public Integer codigo;
    public Integer quantidade;
    public Character tipo;
    public Boolean sucesso = true;
}
```

A classe será utilizada pelo controlador com o nome **MovimentoService**, que será criado no pacote **com.persistencia.controller**:

java

```
@RestController
@EnableTransactionManagement
public class MovimentoService {
    @Autowired
    ProdutoDAO prodDAO;
    @Autowired
    MovimentoDAO dao;

    @GetMapping("/movimento/{cod_prod}")
    public List obterMovimentos(
        @PathVariable(value="cod_prod") Integer codigo){
        return prodDAO.findById(codigo).get().getMovimentos();
    }

    @PostMapping("/movimento")
    public MovimentoVO incluirMovimento(
        @RequestBody MovimentoVO dados){
        Produto produto = prodDAO.findById(dados.codigo).get();
        if((dados.tipo=='S')&&(dados.quantidade>produto.getQuantidade()))
            dados.sucesso=false;
        else {
            dados.sucesso=true;
            produto.setQuantidade(produto.getQuantidade()+
                dados.quantidade*(dados.tipo=='S'?-1:1));
            Movimento movimento = new Movimento();
            movimento.setDia(new Date());
            movimento.setProduto(produto);
            movimento.setQuantidade(dados.quantidade);
            movimento.setTipo(dados.tipo);
            prodDAO.save(produto);
            dao.save(movimento);
        }
        return dados;
    }

    @DeleteMapping("/movimento/{cod_prod}/{id_mov}")
    public MovimentoVO excluirMovimento(
        @PathVariable(value="cod_prod") Integer codigo,
        @PathVariable(value="id_mov") Integer id){
        Produto produto = prodDAO.findById(codigo).get();
        Movimento movimento = dao.findById(id).get();

        MovimentoVO dados = new MovimentoVO();
        dados.codigo = codigo;
        dados.quantidade = movimento.getQuantidade();
        dados.tipo = movimento.getTipo();

        if((dados.tipo=='E')&&(produto.getQuantidade()
```

Observe que um movimento de saída deve ser executado apenas se existirem produtos suficientes, e uma entrada só poderá ser excluída se não tornar a quantidade de produtos negativa.

Note que temos dois gestores para persistência no serviço: o principal é um **MovimentoDAO**, e o secundário é do tipo **ProdutoDAO**, já que lidamos com ambas as tabelas.

O método de consulta **obterMovimentos** responderá no modo **GET**, a partir de endereços no formato **http://localhost:8080/movimento/{cod\_prod}**, onde **cod\_prod** é o código do produto.



## Comentário

Como temos uma relação do tipo OneToMany em Produto, a forma mais simples de obter as movimentações é por meio de getMovimentos, ou seja, a partir do código do produto efetuamos a busca via prodDAO e retornamos seus movimentos.

No caminho **http://localhost:8080/movimento**, em modo **POST**, ocorrerá o redirecionamento para **incluirMovimento**. Nesse caso, deve ser fornecido um **MovimentoVO**, no formato **JSON**, no **corpo** da requisição, tendo como **retorno** os dados de entrada, mas com o indicativo de sucesso ou falha da operação alimentado adequadamente.

A partir do parâmetro **dados**, no método **incluirMovimento**, recuperamos o produto. Se for uma entrada ou existir quantidade suficiente para uma saída, o campo sucesso receberá **true**. Assim:

- uma instância de Movimento é criada, utilizando os dados fornecidos e a data corrente;
- a quantidade de produtos é modificada;
- tanto o produto quanto o movimento são persistidos.

Com relação ao método **excluirMovimento**, que recebe, a partir do endereço de chamada, o código do produto e o **id** da movimentação, ocorrerá o acionamento no modo **DELETE**, para os endereços no formato **http://localhost:8080/movimento/{cod\_prod}/{id\_mov}**.

O produto e o movimento são recuperados da base. Um objeto de **resposta** é instanciado e configurado com os dados da **movimentação**, iniciando-se a verificação da **consistência** em seguida.

Novamente, podemos efetuar os testes através do **Postman**, agora com uma segunda coleção, adotando o nome **Movimentos**.

Iniciaremos testando uma saída, criando a requisição de nome **Inclusão** no endereço **http://localhost:8080/movimento**, modo **POST**, e incluindo no corpo os dados de **MovimentoVO**, no formato **JSON**, com a operação de saída para um produto. Note que o indicador de sucesso é fornecido, mas terá o valor real indicado apenas no retorno. Assim, teremos:

plain-text

```
{
  "codigo": 1,
  "quantidade": 100,
  "tipo": "S",
  "sucesso": true
}
```

Podemos verificar a alteração na quantidade de produtos a partir da chamada **Consulta**, na coleção **Produtos**, ou por meio do **pgAdmin**.

Quanto à movimentação, vamos criar uma requisição **Consulta**, na coleção **Movimentos**, com endereço **http://localhost:8080/movimento/1**, modo **GET**, onde o número ao final representa o código do produto, para que o clique em **Send** retorne todos os movimentos para o produto.



### Atenção

Para testar a remoção do movimento, vamos criar a requisição de nome Exclusão, na coleção Movimentos, apontando para um endereço como http://localhost:8080/movimento/1/101, no modo DELETE.

Segundo a rota, o penúltimo segmento tem o código do produto, e o último tem o **id** do movimento. Mas como esse **id** é gerado automaticamente, será necessário executar a consulta para saber qual número utilizar na chamada.

Agora, temos uma API REST completa para o controle de produtos e movimentações, com base em dois serviços, os quais podem ser utilizados por qualquer tecnologia com suporte ao HTTP, garantindo a **interoperabilidade** de nosso sistema.

# Verificando o aprendizado

Questão 1

Nos bancos de dados relacionais, como o PostgreSQL, a principal característica apresentada é a manutenção de relacionamentos de maneira consistente, com base em chaves estrangeiras, de forma a manter a integridade referencial. Ao nível do JPA, qual anotação define a relação do lado da entidade dependente?

A

@ManyToOne

B

@Entity

C

@OneToMany

D

@Id

E

@ManyToMany



A alternativa A está correta.

Para as entidades JPA, um relacionamento de um-para-muitos deve ser anotado na entidade dependente, como @ManyToOne, com a recepção em uma instância da entidade principal. Já na principal, devemos ter @OneToMany, com uma coleção de dependentes. Com relação às demais, @ManyToMany define um relacionamento de muitos-para-muitos, no qual as duas entidades são fortes, @Entity define uma entidade, e @Id define a chave primária.

Questão 2

Uma API do tipo REST utiliza os métodos do protocolo HTTP e rotas de acesso para a consulta e manipulação de dados, com trânsito de informações preferencialmente no formato JSON. Qual seria o resultado de um acesso ao endereço http:

A

Consulta aos dados do aluno 101

B

Inclusão do aluno 101

C

Consulta aos dados de todos os alunos

D

Exclusão do aluno 101

E

Alteração dos dados do aluno 101



A alternativa E está correta.

Segundo o padrão REST, o acesso via GET se refere às consultas, POST permite a inclusão de uma entidade, PUT altera os dados da entidade, e DELETE efetua a remoção. A rota, formada pelo endereço de base seguido da chave primária, indicaria que o objetivo do acesso seria a alteração dos dados do aluno 101, com os novos dados enviados no corpo da requisição, utilizando o formato JSON.



### 3. Implementação de uma API REST com Spring Data e MongoDB



## Banco de dados MongoDB

### Bases NoSQL

Algo que levou a uma grande popularização das bases NoSQL foi o **Big Data**, já que a manutenção de relacionamentos consistentes pode ser um problema para **grandes massas** de dados divididas em **clusters**.

Adotando bases de dados NoSQL, comprometemos a consistência em favor da disponibilidade, velocidade, escalabilidade e tolerância ao particionamento, ocorrendo grande **heterogeneidade** nas estratégias arquiteturais.

Boa parte das opções não oferece suporte às transações, com as alterações feitas apenas de forma local e repercutidas para os demais nós do cluster posteriormente, após alguns milissegundos.



#### Atenção

Enquanto no modelo relacional definimos esquemas fixos para armazenagem, as estruturas de dados podem ser alteradas dinamicamente em bases NoSQL.



O MongoDB assume uma arquitetura **documental**, na qual uma chave é associada a um documento – que pode ser uma notação de texto, como **XML** e **JSON**, ou algum formato binário, como **PDF**. Os documentos armazenados trazem alguma estrutura de **metadados** associada, levando à classificação dos dados como **semiestruturados**.

## Instalação do MongoDB

Assista ao vídeo e confira os principais pontos sobre este tópico.



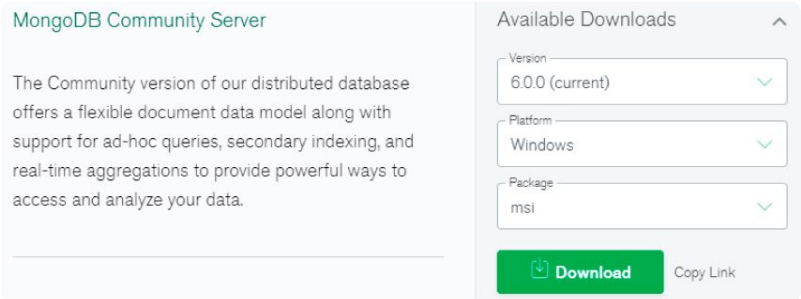
### Conteúdo interativo



Acesse a versão digital para assistir ao vídeo.



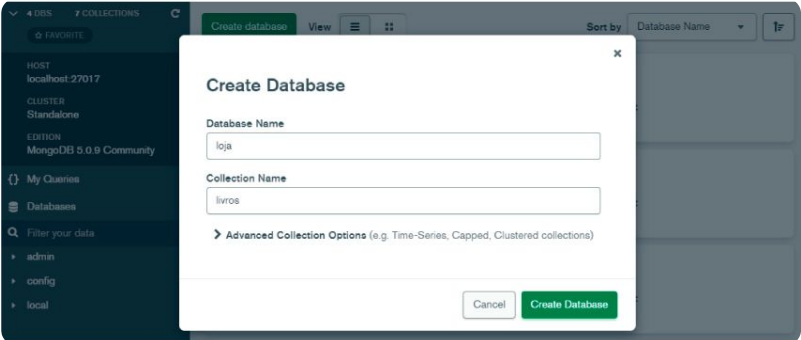
Para trabalharmos com o **MongoDB**, vamos baixar e instalar uma **versão gratuita** no endereço **https://www.mongodb.com/try/download/community**. Escolha a versão adequada:



Download do MongoDB.

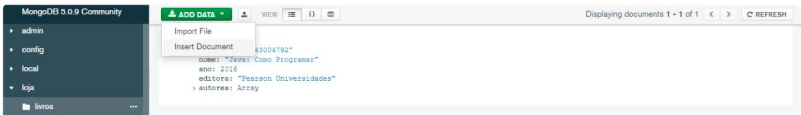
Ao final da instalação, teremos o banco de dados instalado como um **serviço**, além de um aplicativo para gerência com o nome **MongoDB Compass**.

Efetue a conexão e crie o banco de dados **loja**, com a coleção **livros**. Veja:



Criação da base no MongoDB por meio do gerenciador Compass.

Agora, já podemos inserir alguns documentos no formato JSON, navegando até o banco criado na lateral esquerda e escolhendo a coleção **livros**. Após selecionar a coleção, clicamos na opção **Add Data**, seguida de **Insert Document**, abrindo a janela para digitação de documentos, conforme representado a seguir:



Inserção de registro por meio do Compass.

A única restrição acerca dos documentos criados no MongoDB é a necessidade de um campo identificador com o nome **\_id**, que pode ser um número ou texto definido na inserção, ou um identificador **gerado automaticamente**:

```
plain-text

{
  "_id": 101,
  "isbn": "978-8543004792",
  "nome": "Java: Como Programar",
  "ano": 2016,
  "editora": "Pearson Universidades",
  "autores": [
    "Paul Deitel",
    "Harvey Deitel"
  ]
}
```

Podemos efetuar consultas no MongoDB usando a barra de pesquisa, digitando os comandos na sintaxe **JSON**, com base nos operadores disponíveis, e clicando no botão **Find**. A seguir, observe alguns operadores oferecidos pelo MongoDB e a utilização de cada um deles:



\$gt

---

Retorna quando o campo apresenta valor maior que o especificado.



\$lt

---

Retorna quando o campo apresenta valor menor que o especificado.



\$in

---

Verifica se combina com qualquer dos valores do vetor.



\$nin

---

Verifica se não ocorre combinação com qualquer dos valores do vetor.



\$and

---

Combinação lógica de condições com uso de and.



\$or

---

Combinação lógica de condições com uso de or.



\$exists

---

Retorna os documentos que apresentam determinado campo.



\$all

---

Verifica se todos os valores do vetor estão presentes.



# \$size

Verifica se o campo, do tipo vetor, tem o tamanho especificado.

# \$regex

Seleciona os documentos a partir de uma expressão regular.

Como um exemplo simples de pesquisa, poderíamos recuperar todos os livros publicados após o ano 2000, através do operador **\$gt**, desta forma:

```
plain-text
{ano: { $gt: 2000 }}
```

Devido à disposição dos campos na sintaxe JSON, operadores lógicos são utilizados de forma diferente da adotada nas sintaxes mais comuns. Eles são aplicados a vetores de operandos, e não na conexão entre eles. Podemos observar uma consulta mais complexa a seguir:

```
plain-text
{ $or: [
  { $and: [{ ano: { $gt: 2000 } }, { ano: { $lt: 2020 } } ] },
  { autores: { $all: [ "Harvey Deitel", "Paul Deitel" ] } }
]
```

Observe o retorno dos livros que foram publicados após o ano 2000 e antes do ano 2020, ou onde todos os nomes do conjunto (**\$all**) aparecem na lista de autores. Para a restrição do período de publicação, foi utilizado o conector **\$and** (ambas condições verdadeiras), enquanto **\$or** exigirá ao menos um filtro aceito.

# Spring Data e MongoDB

## Criando um projeto Spring Data para o banco de dados MongoDB

Assista ao vídeo e confira os principais pontos sobre este tópico.



# Conteúdo interativo



Acesse a versão digital para assistir ao vídeo.



## Criação do projeto Persistencia004

Visando experimentar o uso de Spring Data para o banco de dados MongoDB, vamos criar o projeto Persistencia004, do tipo Spring Starter Project, com artefato de mesmo nome.

Para o nome do grupo e do pacote, será utilizado o valor com.persistencia. Também devemos incluir as bibliotecas Spring Data MongoDB e Spring Web. Assim, teremos:

Spring Boot Version: 2.7.2

Frequently Used:

☐ H2 Database

☐ PostgreSQL Driver

☐ Spring Data JPA

☒ Spring Data MongoDB

☒ Spring Web

☐ Thymeleaf

Seleção de bibliotecas para o projeto Persistencia004.

Inicialmente, vamos configurar a conexão com o banco de dados MongoDB, por meio do arquivo **application.properties**, no diretório **resources**:

```
spring.data.mongodb.host=localhost
spring.data.mongodb.port=27017
spring.data.mongodb.database=loja
server.port=8081
```

Temos a **conexão** com o MongoDB e a alteração da porta do aplicativo Spring Boot, de 8080 para **8081**, viabilizando sua execução em paralelo com o projeto Persistencia003.

Crie a classe **Livro**, no pacote **com.persistencia.model**, iniciando a construção da camada de persistência:

java

```
@Document(collection = "livros")
public class Livro {
    @Transient
    public static final String SEQUENCE_NAME = "livro_sequence";

    @Id
    public Integer id;
    public String isbn;
    public String nome;
    public Integer ano;
    public String editora;
    public List autores = new ArrayList<>();
}
```

Utilizamos as **coleções** do MongoDB através da anotação **Document**. Da mesma forma que antes, definimos o identificador através da anotação **Id**, mas, agora, adotando o nome fixo **\_id** ao nível do banco.

Também podemos observar um campo estático **transiente**, ou seja, que não é persistido, com o nome **SEQUENCE\_NAME** e valor **livro\_sequence**. Esse campo será utilizado pela estratégia de geração de chaves autoincrementadas, cuja implementação será iniciada com a criação da classe **DataSequence** no pacote **com.persistencia.model**. Logo:

java

```
@Document(collection = "data_sequences")
public class DataSequence {
    @Id
    public String id;
    public Integer atual;
}
```

A nova entidade será persistida na coleção **data\_sequences**, efetuando a gravação do nome da sequência no campo **\_id** e o valor corrente no campo **atual**. De acordo com uma estratégia de autoincremento baseada em sequências, teremos todas elas armazenadas na coleção descrita.

Para operacionalizar o sistema de geração de chaves, precisaremos de um serviço para obtenção e atualização do valor da sequência. Crie a classe **DataSequenceService** e no pacote **com.persistencia.model**:

java

```
@Service
public class DataSequenceService {
    @Autowired
    private MongoOperations mongo;

    public Integer getValor(String seqName){
        DataSequence contador = mongo.findAndModify(
            Query.query(Criteria.where("_id").is(seqName)),
            new Update().inc("atual",1),
            FindAndModifyOptions.options().returnNew(true).upsert(true),
            DataSequence.class);
        return contador.atual;
    }
}
```

Inicialmente, temos a caracterização da classe como um serviço, através da anotação **Service**, e instanciamos um executor de operações do tipo **MongoOperations**, no contexto do Spring. No método **getValor**, tendo como parâmetro o **nome da sequência**, podemos observar o executor sendo utilizado.

A operação **FindAndmodify** buscará o documento no qual **\_id** contém o nome da sequência, com base em um elemento do tipo **CriteriaQuery**, seguido da alteração, via **Update**, incrementando o valor do campo **atual**.



## Comentário

Temos também a opção de geração, em FindAndModifyOptions, quando ainda não existe uma referência para a sequência, e a obtenção como DataSequence. Ao final, teremos o retorno do valor corrente para o campo atual, após a modificação.

O último elemento que criaremos, no pacote **com.persistencia.model**, será a interface com o nome **LivroDAO**, descendendo de **MongoRepository**. Vamos a ela:

java

```
public interface LivroDAO extends MongoRepository{ }
```

# API REST para o banco NoSQL MongoDB

## API REST para o MongoDB

Assista ao vídeo e confira os principais pontos sobre este tópico.



## Conteúdo interativo



Acesse a versão digital para assistir ao vídeo.



Criaremos a API REST por meio da classe **LivroService**, no pacote **com.persistencia.controller**:

java

```
@RestController
@EnableTransactionManagement
public class LivroService {
    @Autowired
    LivroDAO dao;
    @Autowired
    DataSequenceService seqService;

    @DeleteMapping("/livro/{id}")
    public void remover(@PathVariable(value="id") Integer id) {
        dao.deleteById(id);
    }
    @PutMapping("/livro")
    public void alterar(@RequestBody Livro livro) {
        dao.save(livro);
    }
    @PostMapping("/livro")
    public void incluir(@RequestBody Livro livro) {
        livro.id = seqService.getValor(Livro.SEQUENCE_NAME);
        dao.save(livro);
    }
    @GetMapping("/livro")
    public List obterTodos(){
        return dao.findAll();
    }
    @GetMapping("/livro/{id}")
    public Livro obter(@PathVariable(value="id") Integer id){
        return dao.findById(id).get();
    }
}
```

Temos a definição do Web Service com controle transacional. Na sequência, instanciamos um gerenciador **LivroDAO** e um serviço **DataSequenceService** – ambos no contexto do Spring.

Os métodos **remover** e **obter** trabalham com a mesma rota, em que o **id** do livro entra no último segmento do endereço e é capturado no parâmetro de mesmo nome através da anotação **PathVariable**. No entanto, **remover** responde no modo **DELETE**, efetuando a exclusão na base, enquanto **obter** responde a uma chamada **GET**, com o retorno do livro que corresponde ao **id**.

Todas as demais chamadas respondem no endereço de base, sem a inclusão do **id**, com o retorno do conjunto completo de livros no modo **GET**, inclusão no modo **POST** e alteração via **PUT**.

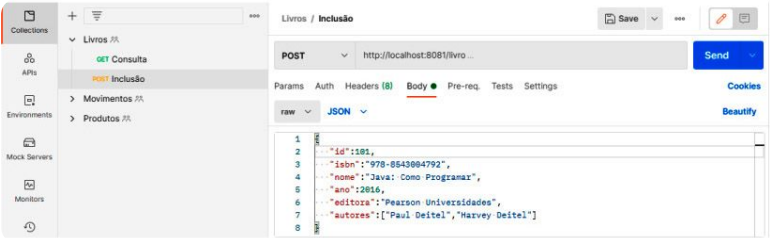


## Comentário

Os métodos incluir e alterar recuperam os dados do livro através de RequestBody, mas, na inclusão, substituímos o id pelo valor autoincrementado, obtido a partir do serviço.

Já podemos testar nossa API, mas, primeiro, precisamos abrir o **MongoDB Compass** e remover os livros adicionados manualmente. Para a remoção, basta passar o mouse sobre o documento e clicar sobre o ícone de uma lixeira que aparecerá no topo.

Vamos abrir o **Postman**, acrescentar uma coleção de requisições com o nome **Livros** e adicionar a ela uma requisição do tipo **POST**, que apontará para **http://localhost:8081/livro**. O corpo da requisição deverá utilizar a configuração **RAW** e **JSON**, adotando o livro de exemplo do tópico acerca do MongoDB, neste mesmo módulo. Veja:



Configuração de chamada para inclusão de livro no Postman.

Executando nosso servidor e clicando no botão **Send** do Postman, teremos a inserção do livro no MongoDB, assumindo para **\_id** o valor **1**. Podemos verificar a criação do documento por meio da ferramenta **Compass**, bem como a definição de uma nova coleção para as sequências, com o nome **data\_sequences**, gerada de forma automática na chamada do serviço.

## Verificando o aprendizado

### Questão 1

O banco de dados MongoDB é um bom exemplo de base NoSQL no modelo documental, com uso do formato JSON, inclusive ao nível dos comandos para busca. Diversos operadores foram definidos para efetuar buscas no ambiente, podendo ser utilizados tanto no modo programado quanto por meio da barra de busca do MongoDB Compass. Qual operador permite retornar os registros que não se encontram em determinado conjunto?

A

\$in

B

\$all

C

\$exists

D

\$nin

E

\$gt



A alternativa D está correta.

O operador para a operação requisitada seria \$nin, equivalente ao NOT IN dos bancos relacionais, pois retorna o registro caso não ocorra a equivalência com os elementos que se encontram no conjunto (vetor). Para o caso contrário, em que há equivalência, teríamos \$in, enquanto \$all exige que todos os valores coincidam, e \$exists verifica a existência do campo. Com relação a \$gt, seria apenas um acrônimo para maior que, ou seja, verifica se o valor do campo é maior que o fornecido para comparação.

Questão 2

Na construção de uma API REST, temos diversas rotas dinâmicas, nas quais os identificadores de uma ou mais entidades são fornecidos através da rota, como nas operações de consulta a partir da chave primária. Qual anotação permite a recuperação do segmento da rota em um parâmetro no método de tratamento ao nível do controlador?

A

@RequestParam

B

@RequestMapping

C

@PathVariable

D

@RequestBody

E

@ModelAttribute



A alternativa C está correta.

Para recuperar os identificadores fornecidos nos segmentos da rota, precisamos usar a anotação `@PathVariable`, podendo ser utilizado mais de um identificador para a mesma rota. Quanto às demais, `@RequestParam` recupera os dados enviados por um formulário padrão, `@RequestBody` recupera os dados do corpo da requisição, `@ModelAttribute` obtém os valores a partir do modelo da página, e `@RequestMapping` define a rota utilizada.



## 4. Aplicação Spring Boot acessando uma API REST



# Biblioteca Retrofit

## Cliente REST com Retrofit

Assista ao vídeo e confira os principais pontos sobre este tópico.



### Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.



## Criação do projeto Persistencia005

Embora seja possível utilizar um objeto de conexão padrão para obtenção dos dados, a adoção de bibliotecas como **Axios**, no ambiente **NodeJS**, e **Retrofit**, para a plataforma **Java**, permite o uso dos métodos do HTTP de forma simplificada. No caso da biblioteca **Retrofit**, ainda temos conversores do formato utilizado na transmissão, como **JSON**, para objetos **Java** ou Kotlin.

Para efetuar um pequeno teste da biblioteca **Retrofit**, vamos criar um projeto padrão console, clicando no menu **File**, seguido de **New** e **Java Project**, o que abrirá o assistente de criação, onde deverão ser executados os seguintes passos:

### Maven Project

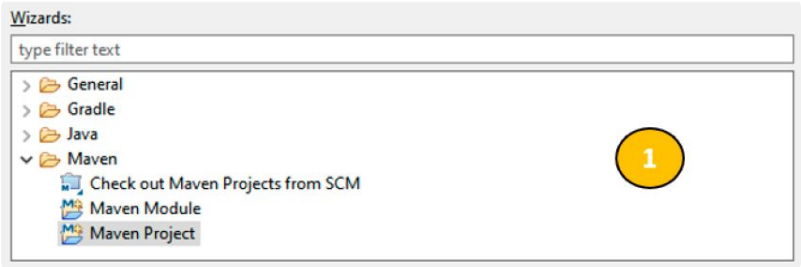
Escolha a opção **Maven Project**, dentro do grupo Maven e

### Finish

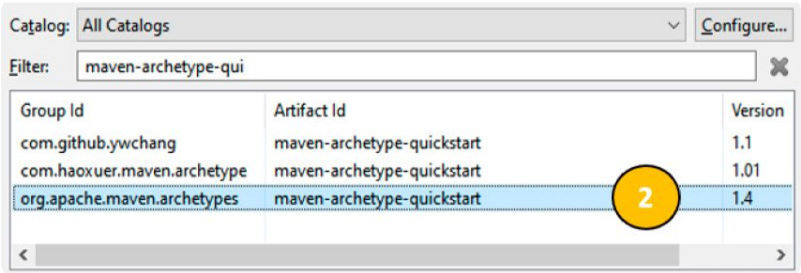
Chegando à última tela do assistente, utilize o valor **persistencia** para o grupo e **Persistencia005** para o artefato, clicando em **Finish** para gerar o projeto.

### maven-archetype-quickstart

Selecione **maven-archetype-quickstart** (apache) como modelo estrutural do projeto e clique em **Next**.

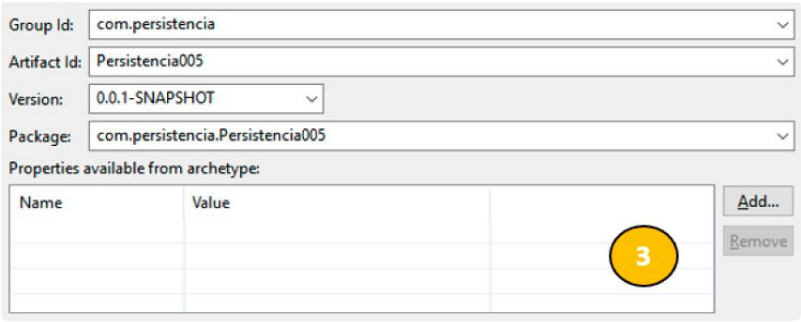


Primeiro passo para a criação de aplicativo Maven padrão.



Segundo passo para a criação de aplicativo Maven padrão.

Observe também o terceiro passo:



Terceiro passo para a criação de aplicativo Maven padrão.

## Inclusão da Biblioteca Retrofit

Agora, crie uma nova versão da entidade **Livro**, no pacote **com.persistencia.model**:

```
java

public class Livro {
    public Integer id;
    public String isbn;
    public String nome;
    public Integer ano;
    public String editora;
    public List autores = new ArrayList<>();
}
```

Depois, você pode criar o cliente para comunicação. Será necessário incluir algumas dependências no arquivo pom.xml na divisão dependencies, onde já teremos o JUnit:

```
plain-text

junit
junit
4.11
test

com.squareup.retrofit2
retrofit
2.9.0

com.squareup.retrofit2
converter-jackson
2.9.0
```

Com a modificação efetuada, incluímos a biblioteca **Retrofit**, bem como o conversor **Jackson**. Esses são produtos de código aberto da **Square Inc** e serão utilizados em nosso cliente.



## Atenção

As dependências também podem ser incluídas com o clique do botão direito sobre o arquivo pom.xml e a escolha da opção Maven, seguida de Add Dependency.

Defina uma interface de acesso aos serviços, com o nome **ILivroService**, no pacote **com.persistencia.service**:

```
java

public interface ILivroService {
    @GET("livro")
    Call< obterTodos();

    @POST("livro")
    Call incluir(@Body Livro livro);

    @DELETE("livro/{id}")
    Call remover(@Path("id") Integer id);
}
```

Tudo que temos é uma **interface** que representa as chamadas necessárias ao serviço, em que cada retorno, incluindo vazio, deve ser encapsulado em um objeto do tipo **Call**. As assinaturas são as mesmas utilizadas no serviço, com o recebimento de um **livro** na inclusão e do **id** na remoção.

As anotações cuidam do resto, delegando para o **Retrofit** a responsabilidade de implementar a chamada. As anotações **GET** e **POST** apontam para a rota principal (**livro**). Já para **DELETE**, há a inclusão do **id** ao final da rota.

Note que temos a anotação **Body**, indicando que o parâmetro **livro** será utilizado no corpo da requisição, enquanto **Path** instrui a utilizar o parâmetro **id** na construção da rota. De forma geral, ocorre uma inversão no sentido da utilização dos parâmetros: no cliente, eles preenchem as informações necessárias e, no servidor, eles capturam os valores enviados.

Agora, especifique o cliente **LivroClient**, no mesmo pacote da interface:

java

```
public class LivroClient {
    private Retrofit retrofit;
    private ILivroService livroService;

    public LivroClient() {
        retrofit = new Retrofit.Builder()
            .baseUrl("http://localhost:8081")
            .addConverterFactory(JacksonConverterFactory.create())
            .build();
        livroService = retrofit.create(ILivroService.class);
    }
    public List obterLivros() {
        try {
            return livroService.obterTodos().execute().body();
        } catch (Exception e) {
            return new ArrayList();
        }
    }
    public boolean incluirLivro(Livro livro) {
        try {
            livroService.incluir(livro).execute();
            return true;
        } catch (Exception e) { return false; }
    }
    public boolean excluirLivro(Integer id) {
        try {
            livroService.remover(id).execute();
            return true;
        } catch (Exception e) { return false; }
    }
}
```

No construtor, temos a configuração do conector **Retrofit**, sendo definido o endereço de **base**, a partir do qual as rotas são calculadas, e o **conversor** adotado na comunicação entre cliente e servidor. Por meio do **JacksonConverterFactory**, ocorre a tradução automática de JSON para Java, na recepção da resposta, bem como no sentido contrário, ao efetuar a requisição.

Aqui, estamos utilizando o modelo síncrono, no qual os métodos da interface cliente são acionados via método **execute**. Note que, para a **inclusão** e para a **alteração**, apenas repassamos os objetos necessários nas chamadas, retornando **true** para uma execução bem-sucedida, ou **false**, no caso contrário, o que é testado através do sistema de exceções padrão do Java.

Com relação à consulta, retornamos o corpo (**body**) da resposta para o chamador, com os dados recebidos em uma coleção Java.

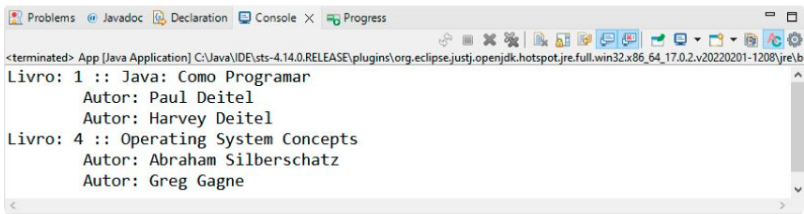
Segundo o modelo utilizado em nosso projeto, teremos uma classe **App**, contendo o código do método **main**, no qual é definido o fluxo de execução. Vamos alterá-lo para o conteúdo seguinte, onde incluímos um livro e listamos o conjunto completo:

java

```
public class App {
    public static void main( String[] args ){
        LivroClient cliente = new LivroClient();
        Livro novo = new Livro();
        novo.nome = "Operating System Concepts";
        novo.ano = 2020;
        novo.autores =
            Arrays.asList("Abraham Silberschatz","Greg Gagne");
        cliente.incluirLivro(novo);
        for(Livro laux: cliente.obterLivros()){
            System.out.println("Livro: " + laux.id + " :: " + laux.nome);
            for(String s: laux.autores)
                System.out.println("\tAutor: "+s);
        }
    }
}
```

Antes de executar nosso novo projeto, devemos iniciar **Persistencia004**, já que será o provedor de informações.

Na sequência, execute **Persistencia005**, escolhendo a opção **Run As**, seguida de **Java Application**, e selecionando a classe **App** para início da execução. Veja o resultado:



Execução do projeto Persistencia005.

Na utilização do **Retrofit**, o encapsulamento do tipo de retorno serve para trabalhar tanto de forma **síncrona**, com a invocação via **execute**, quanto de forma **assíncrona**, através de uma chamada **enqueue** e a criação de um objeto **Callback**. Logo:

```
java

livroService.obterTodos().enqueue(new Callback<>() {
    public void onResponse(Call<> call,
        Response<> response) {
        for (Livro laux: response.body()) {
            System.out.println("Livro: " + laux.id + " :: " + laux.nome);
        }
    }
    public void onFailure(Call<> call, Throwable t) {
        System.out.println("Falha na comunicação");
    }
});
```

Para a implementação do objeto de **Callback**, temos os métodos **onResponse**, no qual a informação recebida é disponibilizada no parâmetro response, e **onFailure**, que possibilita tratar os erros de comunicação ocorridos.

# Padrão Service Locator

## Criação do projeto CamadaIntegracao

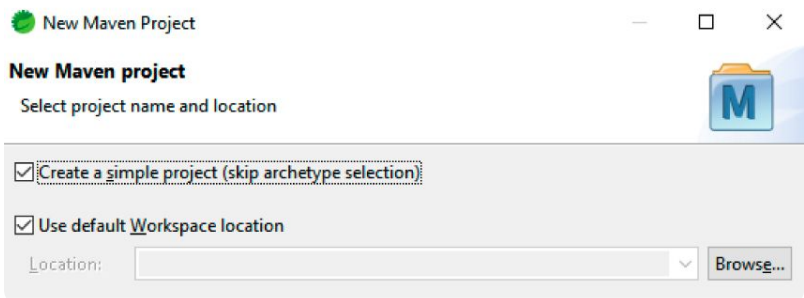
Assista ao vídeo e confira os principais pontos sobre este tópico.

### Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Uma boa prática arquitetural é a separação dos componentes que efetuam a conexão com os servidores da lógica do sistema – algo defendido no padrão **Service Locator**.

Para conseguir essa separação e viabilizar o reuso, vamos criar um projeto, com o nome **CamadaIntegracao**, clicando no menu **File**, seguido das opções **New** e **Project**, além da seleção de **Maven Project**, na divisão **Maven**. **Agimos** da mesma forma como fizemos no projeto Persistencia005, mas, agora, marcando a opção **Create Simple Project**.



Clicando em **Next**, adotaremos o grupo **com.persistencia** na tela seguinte, além do nome do projeto e artefato como **CamadaIntegracao**. Ao clicar em **Finish**, nessa última tela, teremos um projeto criado e modificaremos o arquivo **pom.xml**:

plain-text

```
4.0.0
com.persistencia
CamadaIntegracao
0.0.1-SNAPSHOT
CamadaIntegracao
Modulo de Integracao

com.squareup.retrofit2
retrofit
2.3.0

com.squareup.retrofit2
converter-jackson
2.3.0
```

## Implementação dos pacotes

Tudo que fizemos foi acrescentar as bibliotecas necessárias para utilizar o **Retrofit** e o conversor de **JSON**. Salvando o arquivo, podemos iniciar nossa implementação, a começar pela cópia dos pacotes **com.persistencia.model** e **com.persistencia.service** de **Persistencia005**, incluindo suas respectivas classes e interfaces.



### Atenção

Um erro deverá ocorrer ao nível da entidade Livro, devido ao fato de a inferência de tipo para os genéricos ter sido adicionada em versões mais recentes do compilador. É possível corrigir isso por meio da sugestão oferecida para mudança do nível de código. Essa mudança também pode ser feita com a abertura das propriedades do projeto, a escolha da opção Java Compiler, no menu de navegação esquerdo, e a marcação da opção use compliance from execution environment.

Copie o arquivo **MovimentoVO**, de **Persistencia003**, encontrado no pacote **com.persistencia.controller**, para o pacote **com.persistencia.model** do novo projeto. Crie versões alteradas de Produto e Movimento no pacote **com.persistencia.model**:

java

```
public class Produto {
    private Integer codigo;
    private String nome;
    private Integer quantidade;
    public Produto(){ }
    public Produto(Integer codigo, String nome, Integer quantidade){
        this.codigo = codigo;
        this.nome = nome;
        this.quantidade = quantidade;
    }
    public Integer getCodigo() { return codigo; }
    public void setCodigo(Integer codigo) { this.codigo = codigo; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public Integer getQuantidade() { return quantidade; }
    public void setQuantidade(Integer quantidade) {
        this.quantidade = quantidade;
    }
}
```

A classe **Movimento** adota um acesso mais plano, sem o uso de getters e setters:

java

```
public class Movimento {
    public Integer id;
    public Integer quantidade;
    public Character tipo;
    public Date dia;
}
```

Agora, é possível criar a interface **IProdutoService**, no pacote **com.persistencia.service**, para o acesso aos serviços de Produto:

java

```
public interface IProdutoService {

    @GET("produto")
    Call< obterTodos();

    @GET("produto/{codigo}")
    Call obterProduto(@Path(value="codigo") Integer codigo);

    @POST("produto")
    Call incluir(@Body Produto produto);

    @PUT("produto/{codigo}")
    Call alterar(@Path(value="codigo") Integer codigo,
        @Body Produto produto);

    @DELETE("produto/{codigo}")
    Call remover(@Path(value="codigo") Integer codigo);
}
```

A construção da interface segue os moldes anteriores com:

- encapsulamento em objetos do tipo Call;
- modo como GET ou PUT, especificado por anotação de mesmo nome, juntamente com a rota;
- identificadores recuperados via Path;
- corpo da resposta recuperado com Body.

Usaremos a mesma abordagem para **IMovimentoService**, no pacote **com.persistencia.service**:

java

```
public interface IMovimentoService {

    @GET("/movimento/{cod_prod}")
    Call< obterMovimentos(
        @Path(value="cod_prod") Integer codigo);

    @POST("/movimento")
    Call incluirMovimento(@Body MovimentoVO dados);

    @DELETE("/movimento/{cod_prod}/{id_mov}")
    Call excluirMovimento(
        @Path(value="cod_prod") Integer codigo,
        @Path(value="id_mov") Integer id);
}
```

Com as interfaces para os Web Services relacionados a **produtos** e **movimentos** especificadas, crie a classe **ProdutoClient** no pacote **com.persistencia.service**, englobando ambos os serviços. Aqui, estamos concentrando os serviços que usam a porta **8080**, enquanto o serviço encapsulado em **LivroClient** respondia na porta **8081**. Veja:

java

```
public class ProdutoClient {
    private Retrofit retrofit;
    private IProdutoService produtoService;
    private IMovimentoService movimentoService;

    public ProdutoClient() {
        retrofit = new Retrofit.Builder()
            .baseUrl("http://localhost:8080")
            .addConverterFactory(JacksonConverterFactory.create())
            .build();
        produtoService = retrofit.create(IProdutoService.class);
        movimentoService = retrofit.create(IMovimentoService.class);
    }

    // Início das chamadas relacionadas a Produto
    public List obterProdutos() {
        try {
            return produtoService.obterTodos().execute().body();
        } catch (Exception e) {
            return new ArrayList();
        }
    }

    public Produto obterProduto(Integer codigo) {
        try {
            return produtoService.obterProduto(codigo).execute().body();
        } catch (Exception e) {
            return null;
        }
    }

    public boolean incluirProduto(Produto produto) {
        try {
            produtoService.incluir(produto).execute();
            return true;
        } catch (Exception e) { return false; }
    }

    public boolean alterarProduto(Integer codigo, Produto produto) {
        try {
            produtoService.alterar(codigo, produto).execute();
            return true;
        } catch (Exception e) { return false; }
    }

    public boolean excluirProduto(Integer codigo) {
        try {
            produtoService.remover(codigo).execute();
            return true;
        } catch (Exception e) { return false; }
    }

    // Início das chamadas relacionadas a Movimento
    public List obterMovimentos(Integer codProduto) {
        try {
            return movimentoService.obterMovimentos(codProduto)
                .execute().body();
        } catch (Exception e) {
            return new ArrayList();
        }
    }

    public MovimentoVO incluirMovimento(MovimentoVO dados) {
        try {
            return movimentoService.incluirMovimento(dados)
                .execute().body();
        } catch (Exception e) {
            dados.sucesso = false;
            return dados;
        }
    }

    public MovimentoVO excluirMovimento(Integer codProduto, Integer id){
        try {
            return movimentoService.excluirMovimento(codProduto, id)
                .execute().body();
        } catch (Exception e) {
            MovimentoVO aux = new MovimentoVO();
            aux.codigo = 0;
            aux.sucesso = false;
            return aux;
        }
    }
}
```

Temos um objeto **Retrofit** apontando para a porta **8080**, e subsequente obtenção das instâncias de acesso baseadas em **IProdutoService** e **IMovimentoService**. As chamadas para o Web Service de produtos são similares às que foram utilizadas no cliente anterior, com a substituição da classe **Livro** pela classe **Produto**.

Com relação às chamadas efetuadas via **IMovimentoService**, podem ser observadas algumas diferenças, como o retorno do indicador de **sucesso** por meio de um **MovimentoVO**, decorrente do comportamento do próprio Web Service.



### Comentário

O método `excluirMovimento`, quando executado corretamente ao nível do servidor, retornará sucesso ou falha em decorrência de restrições que podem ocorrer durante o processamento, enquanto uma falha de comunicação causar uma exceção, levando ao retorno de um `MovimentoVO` com código zero e campo sucesso indicando a falha.

Já o método **incluirMovimento** retornará, em decorrência da falha de comunicação, os próprios valores originais, mas com o campo sucesso alterado para **false**, indicando a falha.

Finalmente, o método **obterMovimentos**, que recebe o código do produto e retorna a lista com seus movimentos, retornará uma lista vazia quando uma falha de comunicação ocorrer.

## Criação da interface web

### Criação do projeto PersistenciaFinal

Assista ao vídeo e confira os principais pontos sobre este tópico.



### Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Crie o projeto da **interface de usuário**, com o nome **PersistenciaFinal**, artefato de mesmo nome, além de grupo e pacote básico denominados **com.persistencia**.

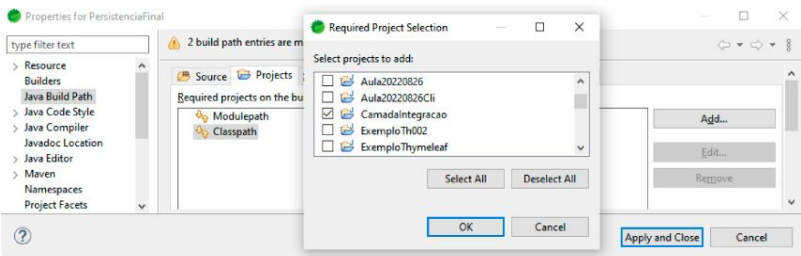


## Comentário

Já que nossa arquitetura utiliza Web Services RESTful e a camada de integração encapsulou as chamadas necessárias, não incluiremos elementos relacionados à persistência no novo projeto. Será necessária apenas a seleção das bibliotecas Spring Web e Thymeleaf.

Inclua **CamadaIntegracao** como dependência de **PersistenciaFinal**: clique com o botão direito no novo projeto e escolha a opção **Properties**. Na janela que será aberta, acesse a divisão **Java Build Path**, aba **Projects** e elemento **Classpath**, clicando depois no botão **Add**.

Uma nova janela será aberta. Selecione **CamadaIntegracao**. Com o retorno para a janela de propriedades, clique em **Apply and Close**, para que a inclusão da dependência seja efetiva:



Inclusão de projeto como dependência.

## Implementação dos controladores

Agora, defina o controlador **ProdutoController** no pacote **com.persistencia.controller**:

java

```
@Controller
@RequestMapping("/web/produto")
public class ProdutoWebController {

    private ProdutoClient prodc = new ProdutoClient();

    private List obterOrdenado(){
        List lista = prodc.obterProdutos();
        lista.sort(new Comparator() {
            @Override
            public int compare(Produto o1, Produto o2) {
                return o1.getNome().compareToIgnoreCase(o2.getNome());
            }
        });
        return lista;
    }

    @GetMapping("listar")
    public String obterProdutos(Model model) {
        model.addAttribute("produtos", obterOrdenado());
        return "produtoListagem";
    }

    @GetMapping("excluir/{codigo}")
    public String excluirProduto(
        @PathVariable("codigo") Integer codigo, Model model) {
        String mensagem = (prodc.excluirProduto(codigo))?
            "Produto excluído com sucesso":"Erro na exclusão";
        if(prodc.obterProduto(codigo)!=null)
            mensagem = "A exclusão não é possível quando há movimentos";
        model.addAttribute("mensagem", mensagem);
        model.addAttribute("produtos", obterOrdenado());
        return "produtoListagem";
    }

    private String execSalvar(
        Produto produto, boolean incluir, Model model) {
        boolean sucesso = (incluir)?
            prodc.incluirProduto(produto):
            prodc.alterarProduto(produto.getCodigo(), produto);
        String mensagem = (sucesso)?
            "Dados armazenados com sucesso":"Erro ao salvar os dados";
        model.addAttribute("mensagem", mensagem);
        model.addAttribute("produtos", obterOrdenado());
        return "produtoListagem";
    }

    @PostMapping("salvar")
    public String salvarProduto(
        @ModelAttribute Produto produto, Model model) {
        return execSalvar(produto, true, model);
    }

    @PostMapping("salvar/{codigo}")
    public String salvarProduto(
        @PathVariable("codigo") Integer codigo,
        @ModelAttribute Produto produto, Model model) {
        return execSalvar(produto, true, model);
    }

    @GetMapping("incluir")
    public ModelAndView incluirDados() {
        return new ModelAndView("produtoDados", "produto", new Produto());
    }

    @GetMapping("alterar/{codigo}")
    public ModelAndView alterarDados(
        @PathVariable("codigo") Integer codigo) {
        return new ModelAndView(
            "produtoDados", "produto", prodc.obterProduto(codigo));
    }
}
```

Inicialmente, temos uma instância de **ProdutoClient**, o método privado **obterOrdenado**, que recupera os produtos a partir da chamada para **obterProdutos**, ordenando o resultado através do método **sort**, e um **Comparator**. Esse método será chamado quando o template de destino for **produtoListagem**, no qual os produtos serão exibidos em uma tabela HTML.

Na rota **web/produto/listar**, em modo **GET**, teremos apenas a definição do atributo **produtos**, com base na coleção ordenada de produtos, enquanto a rota **web/produto/excluir**, acrescida do código do produto, efetuará sua exclusão.



## Comentário

Note que `excluirProduto` conseguirá detectar apenas erros de conexão. Será necessária uma consulta, após a tentativa de exclusão, para verificar se a integridade referencial não impediu a remoção, de forma a obter a mensagem correta. Em ambos os casos, teremos como destino o template `produtoListagem`.

O método privado `execSalvar` concentra as ações comuns de persistência para inclusão e alteração, em que o parâmetro `incluir` define se `incluirProduto` ou `alterarProduto` será invocado, tendo o resultado da execução armazenado em `sucesso` com o objetivo de definir a `mensagem` correta para o usuário.

A inclusão é efetuada na rota `web/produto/salvar` enquanto a alteração segue a mesma rota, com o acréscimo do código do produto – ambos os casos no modo `POST`, tendo como destino o template `produtoListagem`. Note como `ModelAttribute` é utilizado para recuperar os dados do produto, enviados a partir de formulário adequado.

Para inclusão de dados, foi definida a rota `web/produto/incluir`, com uso de um produto vazio no modelo, enquanto a rota `web/produto/alterar`, seguida do código do produto, o pesquisará o e o utilizará no modelo. Em ambos os casos, é usado o modo `GET`, direcionando para o template `produtoDados`.

O segundo controlador é a classe `MovimentoWebController`, no pacote `com.persistencia.controller`:

java

```
@Controller
@RequestMapping("/web/produto/movimento")
public class MovimentoWebController {
    private ProdutoClient prodc = new ProdutoClient();

    private List obterOrdenado(Integer codigo){
        List lista = prodc.obterMovimentos(codigo);
        lista.sort(new Comparator() {
            @Override
            public int compare(Movimento o1, Movimento o2) {
                return o2.dia.compareTo(o1.dia);
            }
        });
        return lista;
    }
    @GetMapping("listar/{codigo}")
    public String obterMovimentos(
        @PathVariable("codigo") Integer codigo, Model model) {
        model.addAttribute("produto", prodc.obterProduto(codigo));
        model.addAttribute("movimentos", obterOrdenado(codigo));
        return "movimentoListagem";
    }
    @PostMapping("salvar/{codigo}")
    public String salvarMovimento(
        @PathVariable("codigo") Integer codigo,
        @RequestParam("quantidade") Integer quantidade,
        @RequestParam("tipo") Character tipo, Model model) {
        MovimentoVO movimento = new MovimentoVO();
        movimento.codigo = codigo;
        movimento.quantidade = quantidade;
        movimento.tipo = tipo;
        movimento.sucesso = false;
        MovimentoVO resultado = prodc.incluirMovimento(movimento);
        String mensagem = (resultado.sucesso)?
            "Movimentação efetuada":"Erro na movimentação";
        model.addAttribute("mensagem", mensagem);
        model.addAttribute("produto", prodc.obterProduto(codigo));
        model.addAttribute("movimentos", obterOrdenado(codigo));
        return "movimentoListagem";
    }
    @GetMapping("excluir/{codigo}/{id}")
    public String excluirMovimento(
        @PathVariable("codigo") Integer codigo,
        @PathVariable("id") Integer id, Model model) {
        MovimentoVO resultado = prodc.excluirMovimento(codigo,id);
        String mensagem = (resultado.sucesso)?
            "Exclusão efetuada":"Erro na exclusão";
        model.addAttribute("mensagem", mensagem);
        return obterMovimentos(codigo,model);
    }
}
```

Para a movimentação do produto, teremos apenas o template **movimentoListagem**, contendo tanto o formulário de inclusão quanto a exibição do conjunto em tabela. Ele receberá, como atributos do modelo, um **produto**, os **movimentos** relacionados a ele e a **mensagem** informativa opcional, utilizada ao concluir as operações de inclusão e exclusão.

Assim como para produtos, temos um método privado para obter os movimentos já ordenados a partir do mais recente. Com base na rota **web/produto/movimento/listar**, com o **código** do produto no modo **GET** temos a definição dos atributos para o **produto** e o conjunto ordenado de **movimentos**, seguido do direcionamento para o template.



## Atenção

A inclusão do movimento ocorre na rota `web/produto/movimento/salvar`, acrescida do código do produto no modo POST, com os dados recuperados via RequestParam: eles são fornecidos a partir de campos individuais.

Com base nos dados fornecidos foi definido um **MovimentoVO**, utilizado na chamada para **incluirMovimento**, em que o campo **sucesso**, do retorno, define a **mensagem** para o usuário.

Finalmente, a exclusão ocorrerá na rota **web/produto/movimento/excluir**, acrescida do **código** do produto e **id** do movimento, no modo **GET**. Novamente, o retorno do método – que, aqui, será **excluirMovimento** – permitirá definir a **mensagem** que será apresentada ao usuário. Como o final das operações de inclusão e exclusão coincide com o processamento da listagem em **obterMovimentos**, ele é invocado ao final de cada uma das operações.

O controlador de **livros** será bastante simples, tendo apenas a rota **web/livro/listar**, onde ocorrem a configuração do atributo **livros** e o retorno do template de exibição (**livroListagem**). Vamos criar a classe **LivroWebController** no pacote **com.persistencia.controller**:

```
java
@Controller
@RequestMapping("/web/livro")
public class LivroWebController {

    private LivroClient livroc = new LivroClient();

    @GetMapping("listar")
    public String obterLivros(Model model) {
        model.addAttribute("livros", livroc.obterLivros());
        return "livroListagem";
    }
}
```

Também precisaremos de um controlador para a **página inicial**, correspondendo à execução do template **index**. Teremos a classe **HomeWebController** no pacote **com.persistencia.controller**, conforme a seguinte listagem, na qual direcionamos a rota raiz para o template:

```
java
@Controller
public class HomeWebController {
    @GetMapping("/")
    public String showHome() {
        return "index";
    }
}
```

Como o novo aplicativo deverá executar em paralelo aos servidores REST, ele deve ocupar uma porta diferente. Por isso, no diretório **resources** é necessário alterar o arquivo **application.properties** para o seguinte conteúdo, de forma a escolher a porta **8082**:

server.port=8082

A configuração final de nossa arquitetura será:

Persistencia003

Porta: 8080  
Característica: API REST para gerenciamento de produtos, com persistência no banco de dados PostgreSQL.



## Persistencia004

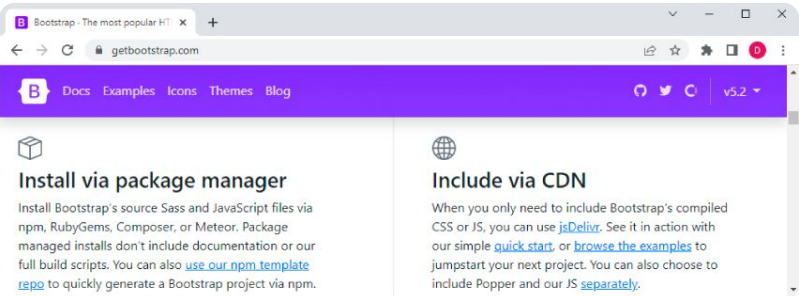
Porta: 8081  
Característica: API REST para gerenciamento de livros, com persistência através do banco de dados MongoDB.

## PersistenciaFinal

Porta: 8082  
Característica: Sistema cadastral no modelo Web, comunicando-se com as APIs por intermédio do projeto **CamadaIntegracao**.

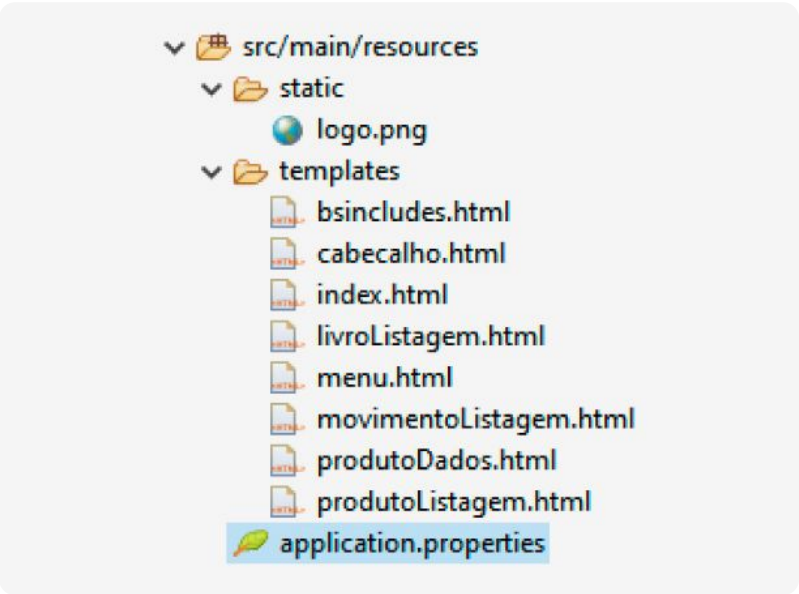
## Utilização do Bootstrap

O framework **Bootstrap** facilita muito a definição do design para páginas HTML. Acesse <https://getbootstrap.com>:



Página oficial do Bootstrap.

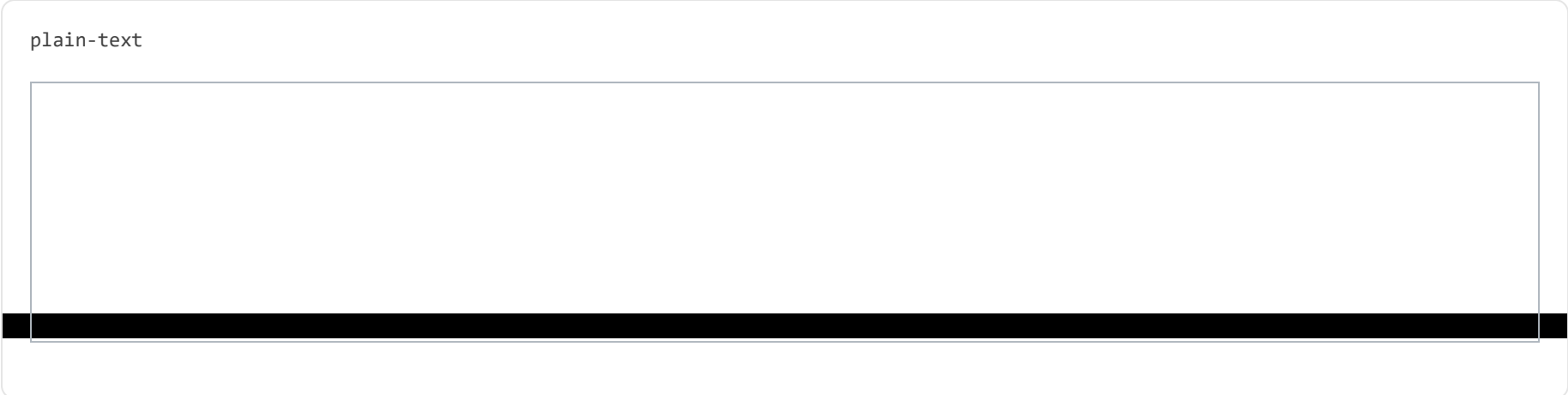
No projeto PersistenciaFinal, vamos definir os templates HTML de nosso sistema cadastral com as características gráficas configuradas via Bootstrap. Os modelos serão criados no diretório templates, a partir de resources. Ao final, obtemos a seguinte configuração:



Configuração final dos resources de PeristenciaFinal.

## Utilização do Thymeleaf

Como o Thymeleaf permite definir fragmentos reutilizáveis, começaremos criando os modelos que serão incluídos nos templates de resposta. O primeiro fragmento, com o nome inclusoes, será definido no arquivo bsincludes.html, com a inclusão de todas as bibliotecas necessárias:



## Página inicial

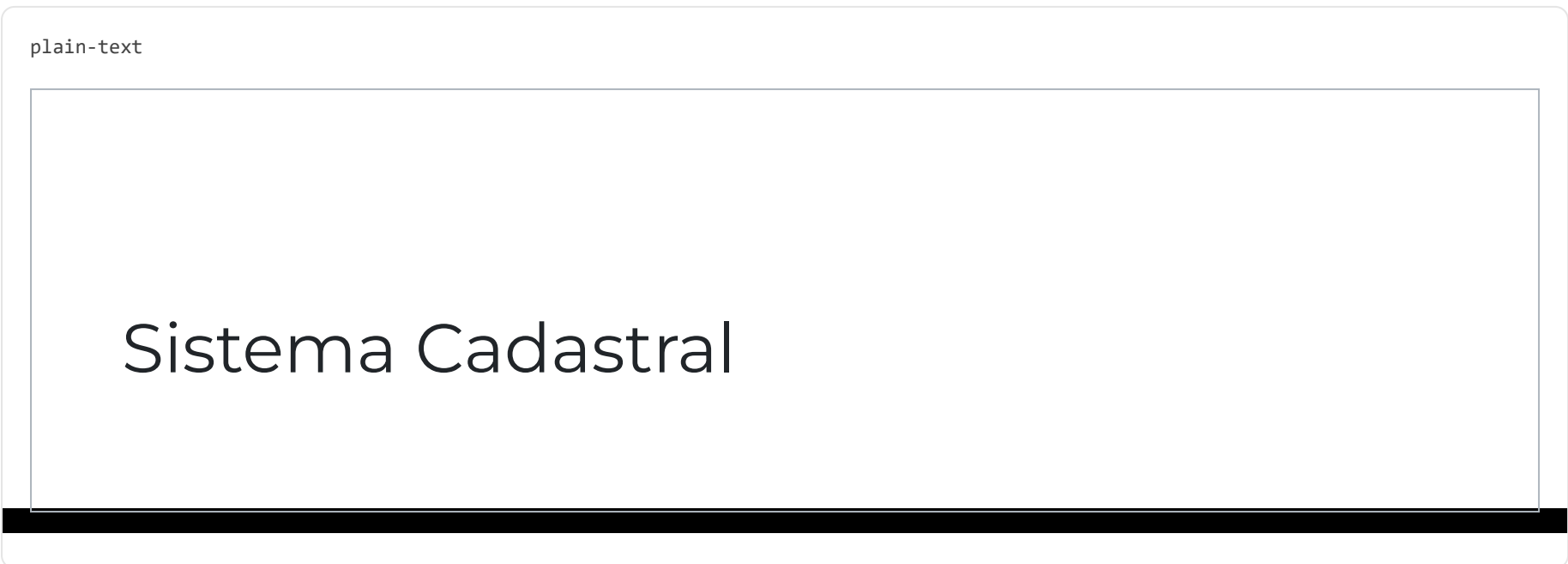
Outro fragmento será o **menuprincipal**, no arquivo **menu.html**, conforme codificação:



Nosso menu se baseia na tag **nav**, utilizando a formatação escura (**bg-dark**) do Bootstrap. Os links usarão **nav-item** e **nav-link**. Temos caminhos para raiz, listagem de produtos e listagem de livros, sempre com correção via **arroba** no atributo **th:href**.

Observe o acréscimo dinâmico de classes CSS via **th:classappend**, a partir do **índice** fornecido para o fragmento, deixando ativo o menu referente à página corrente.

Falta o **cabecalho**, definido no arquivo **cabecalho.html**:



Como logotipo, podemos utilizar uma figura de sua escolha, do tipo **PNG**, renomeada para **logo.png** e colocada no diretório **static** a partir de **resources**. O objetivo desse diretório é fornecer recursos de forma pública no site, considerando a rota raiz como base.

Agora, crie o template **index.html**, definindo a página inicial:

plain-text

# Bem-vindo ao sistema de exemplo

Nosso sistema trabalha com dois servidores REST e  
uma interface no modelo Web

Note o uso de **th:replace** para substituição pelos **fragmentos**, além de uma div, formatada pelo Bootstrap, contendo as informações de abertura do sistema. Para testar, inicie **Persistencia003**, **Persistencia004** e, finalmente, **PersistenciaFinal**. Acesse o endereço **localhost:8082**:



Apresentação da página inicial de PersistenciaFinal.

## Páginas relacionadas aos produtos

Agora, defina o template **produtoListagem.html**:

plain-text

# Produtos do Estoque

valor

×

Incluir Produto

| Id    | Nome  | Quantidade |                                                        |
|-------|-------|------------|--------------------------------------------------------|
| valor | valor | valor      | <div>Alterar</div> <div>Excluir</div> <div>Mover</div> |

A parte inicial da página é similar ao template do index: o título é apresentado no corpo da página. A **div** da sequência é utilizada para exibir a **mensagem**, mas, como nem sempre ela é fornecida, temos uma visualização condicionada via **th:if**.

Adotamos uma classe **alert**, com um botão de fechamento (**data-dismiss**) – tipo de visualização adequada a informações instantâneas.

O botão de inclusão, definido em um **link** comum, é formatado pela classe **btn-primary**. A rota utilizada será **web/produto/incluir**, corrigida com a aplicação de **arroba**.

Finalmente, temos uma tabela onde os títulos são formatados como **table-dark**. O corpo tem uma tag **tr**, que se repete para cada produto da coleção (**th:each**). Os links de alteração, exclusão e movimentação são criados de forma dinâmica, relacionados ao objeto corrente da coleção:



# Sistema Cadastral

Home **Estoque** Biblioteca

## Produtos do Estoque

Produto excluído com sucesso



Incluir Produto

| Id | Nome     | Quantidade |                                                                       |
|----|----------|------------|-----------------------------------------------------------------------|
| 1  | BANANA   | 1300       | <a href="#">Alterar</a> <a href="#">Excluir</a> <a href="#">Mover</a> |
| 3  | LARANJAX | 600        | <a href="#">Alterar</a> <a href="#">Excluir</a> <a href="#">Mover</a> |

Listagem de produtos em PersistenciaFinal.

Clique sobre os botões de inclusão ou alteração. Você será redirecionado para a tela de cadastro, no **template produtoDados.html**:

plain-text

## Dados do Produto

Código

Nome

Quantidade

codigo

Gravar Dados do Produto

Após a parte comum, temos o título da página, a linha divisória e um formulário configurado pelas classes **form** e **container**. Os dados são enviados para a rota **web/produto/salvar**, por meio de **th:action**, com o código adicionado ao final na alteração. A associação ocorre com **produto** via **th:object**, relacionando os campos do formulário às propriedades do objeto.

Na inclusão, há um campo do tipo **number** para a digitação do código, enquanto a alteração usa um campo **hidden**, com alternância controlada por **th:if**.

As classes **form-group** e **form-control** são utilizadas para estilização, e o atributo **required** torna o preenchimento obrigatório:



Dados do Produto

Código

5

Nome

Quantidade

Gravar Dados do Produto

Preencha este campo.

Cadastro de produtos em PersistenciaFinal.

Páginas relacionadas às movimentações

Para a movimentação, implemente `movimentoListagem.html`:

plain-text

# Movimentação do Produto

Produto: valor  
:: Quantidade Atual:

valor

x

Quantidade

Entrada

Saída

Executar Movimentação

| Id    | Data/Hora | Quantidade | Tipo             |                    |
|-------|-----------|------------|------------------|--------------------|
| valor | valor     | valor      | Entrada<br>Saída | <div>Excluir</div> |

Após a parte comum e o título, temos a apresentação dos dados, seguida do mesmo painel de apresentação de **mensagens** da listagem de produtos.

O formulário usa **form-inline** e envia os dados para **web/produto/movimento/salvar** junto ao **código** do produto, no modo **POST**, mas não utiliza um objeto associado. Os elementos **radio** são formatados pela classe **form-check-input**, enquanto os textos usam **form-check-label**. O acréscimo de **required** em um deles exigirá a escolha de uma opção.

No segmento final, há uma tabela para listagem dos movimentos, na qual a exibição do tipo de movimento, como **Entrada** ou **Saída**, foi condicionada ao valor do campo via **th:if**. O link de exclusão usa dois parâmetros relacionados ao **código** do produto e ao **id** do movimento:



# Sistema Cadastral

Home   **Estoque**   Biblioteca

## Movimentação do Produto

Produto: LARANJA :: Quantidade Atual: 600

Quantidade  ☒ Entrada ☐ Saída

! Selecione uma das opções.

| Id         |                              |     |       |                                        |
|------------|------------------------------|-----|-------|----------------------------------------|
| Data/Hora  |                              |     |       |                                        |
| Quantidade |                              |     |       |                                        |
| Tipo       |                              |     |       |                                        |
| 106        | Wed Sep 07 19:58:23 BRT 2022 | 100 | Saída | <input type="button" value="Excluir"/> |

Listagem de movimentos em PersistenciaFinal.

## Páginas relacionadas aos livros

Agora, só falta o template `livroListagem.html`:

python

# Livros da Biblioteca

Quantidade atual 

valor

ISBN: valor

Ano: valor

Para os livros, temos apenas uma exibição no formato cards, na qual o conjunto fica em uma div com classe **card-columns**. Cada ocorrência em uma div é configurada como **card**, repetida pelo **th:each**.

Antes do grupo de cards, com os dados dos livros, há uma div configurada como **badge**, em que é exibida a quantidade total de livros, obtida pelo método **size**. O valor é ressaltado com o uso da classe **badge-light**, enquanto o restante do painel adota **badge-warning**. O **menu**, por meio do valor de **índice**, indica a tela corrente.



## Livros da Biblioteca

Quantidade atual **2**

|                                                                                                                     |                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Java: Como Programar</b><br>ISBN: 978-8543004792<br>Ano: 2016                                                    | <b>Operating System Concepts</b><br>ISBN:<br>Ano: 2020                                                                    |
| <i>Autores:</i> <ul style="list-style-type: none"><li>• <i>Paul Deitel</i></li><li>• <i>Harvey Deitel</i></li></ul> | <i>Autores:</i> <ul style="list-style-type: none"><li>• <i>Abraham Silberschatz</i></li><li>• <i>Greg Gagne</i></li></ul> |

Listagem de livros em PersistenciaFinal.

# Verificando o aprendizado

### Questão 1

A comunicação em rede envolve o conhecimento acerca do protocolo, necessitando do tratamento em classes de fluxo de dados, como InputStream e OutputStream, além de muitas conversões para formatos como JSON e XML. Apesar disso, há bibliotecas que facilitam a implementação desse tipo de tarefa. Qual biblioteca Java permite criar clientes para APIs REST de forma simples, com base em anotações?

A

Thymeleaf

B

Bootstrap

C

Axios

D

Retrofit

E

Express



A alternativa D está correta.

Na biblioteca Retrofit, a comunicação é determinada por uma interface com anotações para as chamadas que serão efetuadas, com o objeto concreto definido a partir da própria biblioteca, eliminando a necessidade de programações mais complexas. Com relação às demais opções, Thymeleaf permite definir modelos HTML que serão preenchidos a partir dos dados enviados pelo controlador, e Bootstrap facilita a formatação das interfaces de usuário do tipo HTML. Tanto Axios quanto Express são ferramentas para o NodeJS, assumindo papéis similares ao Retrofit e ao Spring Boot.

Questão 2

Com a grande diversidade de plataformas da atualidade, os requisitos de responsividade são essenciais, necessitando de algum esforço em termos de organização de folhas de estilos e boa compreensão acerca dos elementos de usabilidade. Esse esforço pode ser minimizado pelo uso de recursos como o Bootstrap, que define a formatação de maneira simples, baseada em classes CSS. Qual classe deve ser utilizada em um botão para exclusão de registro?

A

btn-danger

B

btn-primary

C

btn-success

D

btn-info

E

btn-warning



A alternativa A está correta.

As classes citadas definem cores para os botões de acordo com o padrão utilizado no sistema. Nesse caso, devemos utilizar btn-danger, que traz uma cor vermelha, indicando ação perigosa ou irreversível. Quanto às demais opções, btn-primary é a cor padrão, btn-info é usada para informações comuns, btn-warning define uma situação que merece atenção, e btn-success é adotada na conclusão de operações bem-sucedidas.



## 5. Conclusão



# Considerações finais

O framework Spring, adotado no mercado de forma ampla, desponta como uma ferramenta de grande produtividade, por permitir a construção de sistemas Web na arquitetura MVC, com prazos curtos e garantia da qualidade necessária.

Em termos de repositórios de dados, o modelo relacional, no qual o PostgreSQL é uma opção de grande aceitação, ainda obtém a preferência na construção de sistemas cadastrais, mas com a chegada do Big Data, bases NoSQL, como o MongoDB, têm se tornado muito comuns.

A criação de APIs REST, com o trânsito de dados no formato JSON, é uma das opções mais aceitas atualmente, e o uso de Spring Web garante uma implementação simples, integrada aos demais módulos do framework.



## Podcast

Para encerrar, ouça os principais pontos abordados neste estudo.



## Conteúdo interativo

Acesse a versão digital para ouvir o áudio.



## Explore +

Verifique a documentação **Spring Tutorial**, oferecida pelo Baeldung, com uma excelente trilha de aprendizagem para o framework Spring.

Leia o artigo **Análise de desempenho de banco de dados: PostgreSQL padrão e um Cluster utilizando Postgres-BDR**, de Hudson Souza e Roberto Pereira.

Confira o guia **MongoDB CRUD Operations**, na documentação oficial do MongoDB, com uma excelente trilha de aprendizagem para os comandos JSON utilizados no banco de dados.

Acesse o artigo **JPA – Muitos-para-Um (ManyToOne)** e compreenda o relacionamento entre entidades no JPA, a partir de uma sequência de tutoriais e vídeos.

Acesse a **documentação oficial do Bootstrap**, navegue pelo menu lateral e aprenda com os exemplos práticos de utilização dos diversos componentes do framework.

## Referências

BOAGLIO, F. **Spring Boot**. 1. ed. São Paulo: Casa do Código, 2017.

BRADSHAW, S.; BRAZIL, E.; CHODOROW K. **MongoDB: definitive guide**. 3. ed. USA: O'Reilly, 2020.

BURKE, B.; MONSON, R. **Enterprise JavaBeans 3.0**. 5. ed. São Paulo: Pearson, 2007.

COSMINA, I.; HARROP, R.; SCHAEFER, C.; HO, C. **Pro Spring**. 5. ed. USA: Apress, 2017.

DEITEL, H.; DEITEL P. **Java, como programar**. 10. ed. São Paulo: Pearson, 2016.

OBE, R.; HSU, L. **PostgreSQL up & running**. 3. ed. USA: O'Reilly, 2018.

PATEL, N. **Spring 5.0 projects**. 1. ed. Reino Unido: Packt Publishing, 2019.

TURNQUIST, G. **Learning Spring Boot 2.0**. 2. ed. Reino Unido: Packt Publishing, 2017.

VARANASI, B.; BELIDA, S. **Spring REST**. 1. ed. USA: Apress, 2021.

WALLS, C. **Spring in action**. 5. ed. USA: Manning, 2018.

ZABOT, D. **Aplicativos com Bootstrap e angular**. 1. ed. São Paulo: Érica, 2020.

